

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA

Corso di Laurea in Informatica
Strumenti Formali per la Bioinformatica — A.A. 2025/2026

Verso Nuovi Standard nell'Allineamento Multiplo di Sequenze

Deep Reinforcement Learning e Algoritmi genetici

Studenti:

Emanuele Falanga	Maurilio La Rocca	Geraldine Montella
e.falanga3@...	m.larocca31@...	g.montella19@...

Abstract

L'allineamento multiplo di sequenze (MSA) rappresenta una sfida fondamentale e NP-completa nella bioinformatica computazionale. Nonostante l'efficacia dei metodi euristici tradizionali, la loro tendenza a stabilizzarsi in ottimi locali limita spesso la precisione biologica dei risultati. In questo studio presentiamo un framework innovativo che integra la potenza esplorativa degli Algoritmi Genetici Multi-Obiettivo (NSGA-II) con la capacità decisionale locale di agenti di Deep Reinforcement Learning. Abbiamo sviluppato e testato diverse architetture, culminando in **DCNNMSA**, un modello basato su Dueling Deep Q-Networks progettato per ottimizzare dinamicamente i punteggi di Sum of Pairs (SP) e Column Score (CS). I test sperimentali su dataset di sequenze ortologhe dimostrano che DCNNMSA non solo supera le versioni baseline, ma compete efficacemente con software dello stato dell'arte come MAFFT e MUSCLE, superando PASTA e ClustalOmega.

Fisciano(SA), April 13, 2026

Contents

1	Introduzione	3
1.1	Contesto del problema dell'Allineamento Multiplo di Sequenze	3
1.2	Approcci esistenti e loro limitazioni	3
1.3	Motivazioni del presente lavoro	3
1.4	Contributi del lavoro	4
1.5	Organizzazione del documento	4
2	Stato dell'arte	5
2.1	Il Problema dell'Allineamento Multiplo	5
2.2	Approcci Classici	5
2.3	Funzioni di Scoring	6
2.4	Approcci Basati su Machine Learning	7
2.5	GA-DPAMSA: Allineamento Ibrido con Algoritmi Genetici e Deep RL . . .	8
2.6	Direzioni di Intervento	9
3	Data Engineering: Il Miglioramento del Dataset	10
3.1	Motivazioni: Dai Dati Sintetici ai Dati Reali	10
3.1.1	La scelta dei Gruppi Ortologhi (OrthoDB)	10
3.2	Acquisizione e Strutturazione del Dataset	11
3.3	Pulizia e Gestione delle Anomalie (Data Preprocessing)	11
3.4	Data Augmentation e Vettorizzazione (HDF5)	12
3.5	Impatto del Nuovo Dataset sulle Prestazioni della Baseline	12
3.6	Considerazioni Finali e Pipeline di Inferenza	13
4	Architetture Sperimentate	14
4.1	Dilated CNN-based Architectures	14
4.1.1	Introduzione e Motivazioni	14
4.1.2	Dilated Convolutions	14
4.1.3	Double Dueling Q-Network	15
4.1.4	Action Branching	16
4.1.5	Reward system	16
4.1.6	Dilated Convolutions-based architecture	17
4.1.7	Action Branching and DC-based architecture	18
4.1.8	Discussione e lavori futuri	19
4.2	Allineamento One-Shot con PPO e GRPO	22
4.2.1	Proximal Policy Optimization	22
4.2.2	Group Relative Policy Optimization	22
4.2.3	Componenti utilizzati	23
4.2.4	Environment vettorizzato	24
4.2.5	Interpretazione dell'Output	24
4.2.6	Addestramento e Iperparametri	25
4.2.7	Sperimentazione e Risultati	26
4.2.8	Discussione e Lavori Futuri	26
4.3	Sviluppo del Nuovo Orchestratore Basato su NSGA-II	28
4.3.1	Analisi delle criticità della Baseline	28
4.3.2	Ricerca Locale vs Ricerca Globale: Limiti degli Algoritmi Ibridi . .	28
4.3.3	Architettura utilizza e Motivazioni	29

4.3.4	Analisi dell'Implementazione	30
4.3.5	Confronto Prestazionale	34
5	Risultati	35
5.1	Analisi Comparativa delle Architetture Sviluppate	35
5.1.1	Dalla Baseline all'Ottimizzazione Multi-Obiettivo	35
5.1.2	Confronto Reinforcement Learning	35
5.1.3	L'Egemone nel Benchmark: DCNNMSA	36
5.2	Confronto con lo Stato dell'Arte	38
5.2.1	Analisi Columnn Score (CS)	38
5.2.2	Analisi del Sum of Pairs (SP):	38
5.3	Considerazioni Finali	38
6	Conclusioni e Sviluppi Futuri	40
6.1	Conclusioni	40
6.2	Sviluppi Futuri	41

1 Introduzione

1.1 Contesto del problema dell'Allineamento Multiplo di Sequenze

L'Allineamento Multiplo di Sequenze (MSA) è un problema fondamentale in bioinformatica, essenziale per compiti come l'analisi filogenetica, la scoperta di motivi e la predizione della struttura proteica. L'obiettivo dell'MSA è allineare regioni omologhe attraverso molteplici sequenze biologiche, massimizzando la conservazione di motivi funzionalmente o evolutivamente significativi. Tuttavia, l'MSA è classificato come un problema NP-completo, il che significa che le risorse computazionali richieste per trovare un allineamento ottimale crescono esponenzialmente con il numero e la lunghezza delle sequenze. Di conseguenza, approcci euristici e meta-euristici sono spesso impiegati per approssimare allineamenti ottimali in tempi ragionevoli.

1.2 Approcci esistenti e loro limitazioni

I metodi tradizionali (es. ClustalW [21], MAFFT [10], MUSCLE-5 [6]) sono efficienti, ma soffrono spesso del problema degli ottimi locali, in cui gli errori di allineamento iniziali si propagano durante il processo e degradano il risultato finale. I recenti avanzamenti nell'Intelligenza Artificiale hanno introdotto approcci basati su Deep Reinforcement Learning (DRL) per ottimizzare l'accuratezza.

Il lavoro predecessore a questo progetto, GA-DPAMSA [26], ha proposto un framework ibrido che combina Algoritmi Genetici (GA) e DRL per superare le sfide di scalabilità. Nonostante gli evidenti miglioramenti apportati, l'architettura ereditata presentava pesanti colli di bottiglia e limiti strutturali:

- **Assenza di realismo biologico:** l'addestramento veniva effettuato su dati puramente sintetici, impedendo alla rete di apprendere pattern di conservazione reali.
- **Complessità computazionale:** il modulo predittivo basato su *attention* scalava in modo quadratico, mentre l'agente subiva una crescita esponenziale dello spazio delle azioni rispetto al numero di sequenze.
- **Gestione sub-ottimale degli obiettivi:** l'algoritmo genetico tentava di unire in modo forzato metriche in conflitto (Sum-of-Pairs e Column Score) senza un reale criterio di dominanza, portando a convergenze premature e squilibrate.

1.3 Motivazioni del presente lavoro

L'analisi delle criticità della baseline ha evidenziato come il miglioramento dell'MSA richieda un'indagine su più fronti. Il presente lavoro si configura come un'esplorazione sistematica di diverse soluzioni metodologiche e architetturali. L'attività di ricerca nasce dalla volontà di testare sperimentalmente come paradigmi differenti (dalla natura dei dati di addestramento alle architetture neurali fino ai meccanismi di ottimizzazione multiobiettivo) possano interagire per superare i vincoli di scalabilità e fedeltà biologica riscontrati nel lavoro precedente. Questo approccio esplorativo è fondamentale per identificare quali componenti siano realmente determinanti per un salto prestazionale nel dominio biologico reale.

1.4 Contributi del lavoro

Questo progetto estende e rivoluziona il precedente framework introducendo i seguenti contributi chiave:

1. Data Engineering su Dati Reali

- Transizione da sequenze sintetiche a gruppi ortologi reali (Mammalia) estratti da OrthoDB v12.
- Implementazione di una robusta pipeline di preprocessing e vettorizzazione in formato HDF5 ad alte prestazioni.

2. Nuove Architetture di Reinforcement Learning

- Sostituzione dell'Attention-Based Encoder con architetture basate su *Dilated Convolutions* (DCNN) per l'apprendimento efficiente di feature locali.
- Risoluzione dello spazio esponenziale delle azioni tramite la strategia di *Action Branching* (BDQ).
- Esplorazione di architetture *One-Shot* tramite gli algoritmi PPO e GRPO per abbattere i tempi di inferenza.

3. Nuovo Orchestratore basato su NSGA-II

- Passaggio a un algoritmo genetico focalizzato sulla dominanza paretiana per gestire come obiettivi ortogonali il Sum-of-Pairs e il Column Score.
- Introduzione di un innovativo operatore di mutazione mirata, in cui l'agente di RL viene impiegato come meccanismo di *local search* per risolvere sottomatrici sub-ottimali.

1.5 Organizzazione del documento

Il documento è strutturato come segue:

- La **Sezione 2** rivede lo Stato dell'Arte, analizzando le soluzioni tradizionali, quelle basate sull'apprendimento automatico e i possibili miglioramenti degli orchestratori.
- La **Sezione 3** descrive il lavoro di Data Engineering, la creazione del nuovo dataset e il "rinascimento" della baseline.
- La **Sezione 4** presenta in dettaglio le Architetture Sperimentate per l'agente RL e lo sviluppo del nuovo orchestratore basato su NSGA-II.
- La **Sezione 5** espone i Risultati sperimentali, analizzando i benchmark e confrontando le metriche ottenute con i tool di riferimento.
- La **Sezione 6** discute le conclusioni tratte dal lavoro e delinea i possibili Sviluppi Futuri.

2 Stato dell'arte

2.1 Il Problema dell'Allineamento Multiplo

L'allineamento multiplo di sequenze (*Multiple Sequence Alignment*, MSA) consiste nel disporre simultaneamente tre o più sequenze biologiche per evidenziare regioni di similarità funzionale, strutturale o evolutiva. Formalmente, date k sequenze $S_1, S_2, \dots, S_k$ su un alfabeto Σ ($\{A, T, C, G\}$ per il DNA), un allineamento è ottenuto inserendo gap ($-$) in modo che tutte le sequenze raggiungano la stessa lunghezza L , con il vincolo che nessuna colonna sia composta esclusivamente da gap. Il risultato è una matrice $M \in (\Sigma \cup \{-\})^{k \times L}$, dove ogni riga corrisponde a una sequenza e ogni colonna a una posizione omologa ipotizzata. L'obiettivo è trovare M che massimizzi una funzione di scoring (Sezione 2.3).

Per due sequenze il problema è risolvibile in tempo polinomiale tramite programmazione dinamica [14], con complessità $O(nm)$. L'estensione a k sequenze richiede la costruzione di un ipercubo k -dimensionale, portando la complessità a $O(n^k)$, proibitiva anche per un numero modesto di sequenze. Wang e Jiang [23] hanno dimostrato che ottimizzare l'allineamento rispetto alla funzione Sum-of-Pairs è NP-hard. Questo risultato implica che ogni metodo pratico di MSA è necessariamente un'approssimazione e che non esiste un criterio assoluto per stabilire l'ottimalità di un allineamento. La ricerca si è quindi orientata verso euristiche capaci di produrre soluzioni di buona qualità in tempi accettabili.

2.2 Approcci Classici

L'intrattabilità del problema esatto ha prodotto diverse famiglie di euristiche, classificabili per strategia di costruzione.

Approcci progressivi. Riducono il problema multiplo a una serie di allineamenti a coppie, guidati da un albero (*guide tree*) che stabilisce l'ordine di aggregazione. ClustalW [21] costruisce l'albero tramite neighbor-joining sulle distanze a coppie. MUSCLE [6] migliora lo schema iterando la costruzione dell'albero e il raffinamento dell'allineamento. MAFFT [10] sfrutta la FFT per identificare rapidamente regioni omologhe, offrendo diversi livelli di accuratezza e velocità. ClustalOmega [19] scala a migliaia di sequenze combinando mBed embedding e HAlign. Il limite strutturale comune è il *greedy commitment*: errori nelle prime fasi di allineamento si propagano irreversibilmente alle successive, degradando la qualità in proporzione alla divergenza delle sequenze.

Approcci consistency-based. T-Coffee [15] calcola preventivamente tutti gli allineamenti a coppie per costruire una libreria di consistenza che guida le scelte locali con informazioni globali. L'accuratezza migliora significativamente su sequenze divergenti, ma la complessità $O(N^2 L^2)$ per la costruzione della libreria ne limita la scalabilità.

Approcci iterativi. Le modalità iterative di MUSCLE e MAFFT rivisitano l'allineamento attraverso cicli di riallineamento parziale, accettando modifiche solo se lo score migliora. Questo mitiga il *greedy commitment*, ma la convergenza resta tipicamente locale e non garantisce il raggiungimento dell'ottimo globale.

Trasversalmente, questi approcci condividono tre limiti che motivano la ricerca di strategie alternative: nessuna garanzia di ottimalità globale, ottimizzazione implicita di

un singolo criterio di scoring senza meccanismi per bilanciare metriche diverse (come SP e CS), e assenza di apprendimento trasferibile tra istanze diverse del problema.

2.3 Funzioni di Scoring

La valutazione di un allineamento si basa su funzioni di scoring che ne quantificano la qualità. Di seguito si formalizzano le due metriche principali adottate nel presente lavoro.

Sum-of-Pairs (SP). Valuta l'allineamento sommando i punteggi di tutte le coppie di caratteri, colonna per colonna:

$$\text{SP}(M) = \sum_{c=1}^L \sum_{i=1}^k \sum_{j=i+1}^k s(M_{i,c}, M_{j,c}) \quad (1)$$

dove $s(a, b)$ è una funzione di punteggio per la coppia (a, b) . Per coppie di residui, s è determinata da una matrice di sostituzione: BLOSUM [8] o PAM [1] per proteine, tipicamente un sistema match/mismatch a punteggio fisso per DNA. Le coppie gap-gap ricevono convenzionalmente punteggio nullo.

Per le coppie residuo-gap, il modello più semplice prevede una penalità lineare costante per ogni posizione. In letteratura esiste anche il modello affine [7], che separa la penalità di apertura (elevata) da quella di estensione (ridotta), riflettendo la tendenza biologica delle indel a essere eventi contigui. Il presente lavoro adotta il modello lineare; l'integrazione del modello affine è discussa tra gli sviluppi futuri.

Column Score (CS). Il Column Score è una metrica binaria per colonna con due formulazioni distinte. Nella versione *reference-based* [22], utilizzata nei benchmark come BALiBASE, misura la frazione di colonne di un allineamento di riferimento M^* riprodotte identicamente nel test:

$$\text{CS}_{\text{ref}}(M, M^*) = \frac{|\{c \in M^* : c \in M\}|}{|M^*|} \quad (2)$$

Nella versione *intrinseca*, che non richiede un riferimento, conta la frazione di colonne in cui tutti i residui sono identici e non sono presenti gap:

$$\text{CS}_{\text{int}}(M) = \frac{|\{c : M_{1,c} = M_{2,c} = \dots = M_{k,c} \in \Sigma\}|}{L_{\text{res}}} \quad (3)$$

dove L_{res} è il numero di colonne contenenti almeno un residuo non-gap. Normalizzare per la lunghezza totale L penalizzerebbe allineamenti con molte colonne di gap, confondendo la qualità dell'allineamento con la lunghezza dell'output.

Relazione tra le metriche. Lo SP valuta la qualità globale attraverso tutte le relazioni a coppie; il CS intrinseco misura la conservazione delle sequenze, non la qualità dell'allineamento (sequenze divergenti produrranno un CS basso anche con un allineamento perfetto). Le due metriche possono essere in conflitto: massimizzare lo SP tende a frammentare i gap, riducendo il CS.

2.4 Approcci Basati su Machine Learning

L'applicazione di tecniche di machine learning al problema dell'MSA ha aperto direzioni alternative rispetto alle euristiche classiche, distinguibili in due famiglie: metodi supervisionati basati su architetture transformer e metodi basati su reinforcement learning.

Approcci transformer-based. L'analogia tra allineamento di sequenze biologiche e traduzione automatica ha motivato l'adozione di architetture sequence-to-sequence:

- **BetaAlign** [3, 4]: primo aligner basato interamente su deep learning. Concatena le sequenze non allineate in un'unica frase e addestra un transformer encoder-decoder a produrre l'allineamento come traduzione. Utilizza un ensemble di transformer addestrati su dati simulati con parametri di indel diversi. Risultati competitivi con MAFFT, MUSCLE e T-Coffee su benchmark simulati, ma con un vincolo di circa 1024 token (~ 10 sequenze di 100 colonne) dovuto alla complessità quadratica della self-attention ($O(L^2)$) [4].
- **Differentiable Smith-Waterman** [16]: versione differenziabile dell'algoritmo di allineamento, integrabile end-to-end in pipeline di deep learning. Apprende una funzione di scoring da embedding transformer ottimizzata con un task a valle. Orientato all'allineamento a coppie più che al caso multiplo.

Approcci basati su reinforcement learning. Una linea parallela ha formulato l'MSA come problema di decisione sequenziale, in cui un agente apprende una policy di inserimento gap tramite interazione con un environment:

- **Primi approcci** [13, 9]: Q-learning e deep Q-network con LSTM applicati all'MSA. Dimostrano la fattibilità dell'approccio ma con limitazioni in convergenza e scalabilità.
- **DQN con negative feedback** [27]: introduce un meccanismo di stabilizzazione del training e un algoritmo di profiling per l'allineamento progressivo, migliorando accuratezza e stabilità.
- **DPAMSA** [12]: il lavoro più rilevante per il presente progetto. Combina deep RL con tecniche di NLP tramite un DQN che integra positional encoding e self-attention. L'allineamento procede colonna per colonna: ad ogni passo l'agente decide quali sequenze avanzano e quali ricevono un gap. Supera diversi tool classici (ClustalW, MUSCLE, MAFFT) in SP e CS su dataset di piccole dimensioni.

Limiti comuni. Nessuno di questi approcci ha ancora raggiunto gli standard di applicabilità dei metodi classici. In sintesi:

- **Scalabilità:** BetaAlign è limitato a ~ 10 sequenze di 100 colonne, DPAMSA a 3 sequenze di 30 nucleotidi. MAFFT e ClustalOmega gestiscono routinariamente migliaia di sequenze di lunghezza arbitraria.
- **Spazio delle azioni:** negli approcci RL stepwise, le azioni possibili crescono esponenzialmente ($2^k - 1$ per k sequenze).

- **Propagazione degli errori:** la formulazione stepwise soffre di *greedy commitment*, analogamente agli approcci progressivi classici.
- **Dipendenza dai dati:** tutti i metodi basati su apprendimento dipendono dalla qualità e rappresentatività dei dati di addestramento, aspetto affrontato nella Sezione 3.

2.5 GA-DPAMSA: Allineamento Ibrido con Algoritmi Genetici e Deep RL

A partire da DPAMSA, è stato proposto GA-DPAMSA [26], il diretto predecessore del presente lavoro. Si tratta di un framework ibrido che integra un algoritmo genetico (GA) con l'agente di deep reinforcement learning. L'intuizione centrale è separare la ricerca globale nello spazio degli allineamenti (delegata al GA) dalla ricerca locale nelle sotto-regioni critiche (delegata all'agente RL), combinando i vantaggi di entrambi gli approcci.

Architettura. Il sistema opera su allineamenti completi, potenzialmente più grandi della finestra operativa dell'agente RL. Il ciclo evolutivo si articola nelle seguenti fasi:

1. **Inizializzazione:** la popolazione è generata a partire dalle sequenze originali tramite clonazione e inserimento stocastico di gap.
2. **Valutazione:** ogni individuo (allineamento candidato) è valutato in termini di SP e CS.
3. **Selezione:** gli individui migliori sono selezionati per la riproduzione.
4. **Crossover orizzontale:** scambia intere righe (sequenze allineate) tra coppie di genitori.
5. **Mutazione guidata dall'agente:** la sotto-regione con il punteggio peggiore viene identificata, estratta come sub-board delle dimensioni della finestra operativa dell'agente ($k \times W$, es. 3×30), e riallineata dall'agente RL. La sub-board ottimizzata viene reinserita nell'allineamento.

Scalabilità tramite sub-board. L'aspetto chiave di GA-DPAMSA è che l'agente RL opera esclusivamente su sotto-regioni di dimensione fissa, mentre il GA gestisce l'allineamento globale. Questo consente di allineare problemi di dimensione arbitraria (es. 6 o più sequenze di lunghezza variabile) utilizzando un agente addestrato su una scala ridotta (es. 3×30), senza necessità di riaddestramento. Nei benchmark, il sistema ha dimostrato di superare sia DPAMSA che diversi tool classici, confermando l'efficacia dell'approccio ibrido.

Limitazioni. Nonostante i risultati, GA-DPAMSA presenta limiti che motivano i contributi del presente lavoro:

- **Encoder basato su attention:** la Q-network ereditata da DPAMSA utilizza self-attention, che scala quadraticamente rispetto alla lunghezza delle sequenze. Questo vincola le dimensioni della finestra operativa e i tempi di training.

- **Spazio delle azioni esponenziale:** la formulazione stepwise mantiene $2^k - 1$ azioni possibili per colonna, limitando la scalabilità nel numero di sequenze per sub-board.
- **Dati di addestramento sintetici:** l'agente originale è addestrato su sequenze generate casualmente, prive di vincoli evolutivi realistici (Sezione 3).
- **Ottimizzazione mono-obiettivo:** il GA originale ottimizza SP e CS separatamente o tramite unione forzata degli indici migliori, senza un reale criterio di dominanza multi-obiettivo (Sezione 4.3).

Ciascuna di queste limitazioni corrisponde a un asse di intervento del presente lavoro, descritto nelle sezioni successive.

2.6 Direzioni di Intervento

L'analisi delle limitazioni di GA-DPAMSA ha permesso di individuare quattro direzioni di miglioramento, che rappresentano assi di ricerca complementari:

1. **Miglioramento dell'agente locale.** L'agente RL ereditato da DPAMSA presenta limiti sia in scalabilità (encoder basato su attention quadratica, spazio delle azioni esponenziale) che in qualità dell'allineamento prodotto (formulazione stepwise con propagazione degli errori, addestramento su dati sintetici). Questa direzione comprende la sostituzione dell'encoder con architetture più efficienti, la riformulazione dello spazio delle azioni, l'adozione di paradigmi di ottimizzazione della policy più moderni, e la transizione a dati di addestramento biologicamente realistici.
2. **Miglioramento dell'orchestratore genetico.** Le capacità dell'algoritmo genetico possono essere estese valutando approcci quali l'integrazione di meccanismi di ricerca locale, come i Gradient-based Genetic Algorithms (GGA) [5], o l'adozione di architetture multi-obiettivo come l'NSGA-II [2]. Quest'ultimo consente di ottimizzare simultaneamente il Sum-of-Pairs e il Column Score trattandoli come obiettivi ortogonali sulla frontiera di Pareto.
3. **Orchestrazione globale alternativa.** In prospettiva, il GA potrebbe essere sostituito o affiancato da strategie di orchestrazione diverse, come beam search, metaeuristiche ibride, o approcci cooperativi multi-agente. Questa direzione non è stata esplorata nel presente lavoro, ma rappresenta uno sviluppo futuro naturale.
4. **Condizionamento dell'inferenza locale con informazioni globali.** L'agente attualmente opera sulla sub-board senza contesto sull'allineamento circostante. Fornire informazioni esterne alla finestra (ad esempio, profili di consenso delle regioni adiacenti o statistiche globali dell'allineamento) come condizionamento potrebbe migliorare la coerenza tra l'ottimizzazione locale e la qualità globale. Anche questa direzione è identificata come sviluppo futuro.

Tra le varie direzioni di sviluppo appena descritte, in questo progetto abbiamo scelto di esplorare principalmente le prime due. Essendo quelle più vicine al lavoro precedente, rappresentano il punto di partenza ideale per costruire una base solida, indispensabile per poter affrontare in futuro sviluppi più complessi.

3 Data Engineering: Il Miglioramento del Dataset

Uno dei primi e più impattanti interventi di ottimizzazione rispetto all'architettura originale ha riguardato l'individuazione, la definizione e la strutturazione di una nuova pipeline di approvvigionamento dati. Il modello di baseline, infatti, veniva originariamente addestrato e validato su sequenze generate in modo puramente sintetico.

3.1 Motivazioni: Dai Dati Sintetici ai Dati Reali

L'architettura originale generava sequenze nucleotidiche assemblando stringhe casuali di caratteri (A, T, C, G) e iniettando stocasticamente gap e mutazioni. Sebbene questo approccio permetta di generare moli infinite di dati per testare la tenuta del codice, presenta un limite insormontabile per l'addestramento di agenti intelligenti: **l'assenza di vincoli biologici ed evolutivi**.

Un modello di Reinforcement Learning addestrato su dati casuali impara ad allineare il rumore stocastico, ma non sviluppa la capacità di riconoscere i pattern di conservazione reali, i motivi funzionali o le dinamiche di inserzione/delezione (indel) tipiche del DNA. Per far compiere all'agente un salto prestazionale, era imperativo passare a dati biologici reali.

3.1.1 La scelta dei Gruppi Ortologhi (OrthoDB)

Per garantire il massimo realismo biologico, abbiamo progettato una pipeline per il download automatizzato di **geni ortologhi** dal database OrthoDB v12. I geni ortologhi sono geni omologhi presenti in specie diverse, originatisi per speciazione da un singolo gene presente nell'antenato comune. La scelta di questa specifica tipologia di dati non è casuale, ma risponde a tre esigenze fondamentali per il Multiple Sequence Alignment:

- **Conservazione Funzionale:** Essendo derivati da un antenato comune, gli ortologhi mantengono solitamente la stessa funzione biologica. Questo si traduce in regioni di sequenza altamente conservate (motivi) che l'agente deve imparare ad allineare perfettamente.
- **Divergenza Evolutiva Realistica:** Nel corso dei milioni di anni di separazione evolutiva, le sequenze hanno accumulato mutazioni, inserzioni e delezioni fisiologiche. Questo fornisce al modello esempi concreti delle difficoltà di allineamento che incontrerà nel mondo reale, ben diverse dal gap inserito casualmente nel dataset sintetico.
- **Verosimiglianza del Task:** L'allineamento di sequenze ortologhe per studi di filogenesi o genomica comparata è esattamente uno dei task principali per cui i biologi utilizzano tool come ClustalW o MAFFT. Addestrare il modello su questo dominio significa prepararlo per l'utilizzo in produzione.

3.2 Acquisizione e Strutturazione del Dataset

L'acquisizione dei dati è stata automatizzata tramite uno script proprietario (`ortho_data.py`) che interroga le API REST di OrthoDB. Abbiamo focalizzato il download sulle sequenze CDS (Coding DNA Sequences) appartenenti al clade *Mammalia*, garantendo così un grado di parentela evolutiva coerente. Il sistema scarica i file FASTA raggruppati per Orthologous Group (OG), gestendo il caching delle richieste per ottimizzare i tempi di rete e permettendo lo scaling del dataset fino a decine di migliaia di file.

3.3 Pulizia e Gestione delle Anomalie (Data Preprocessing)

I dati biologici grezzi contengono spesso inconsistenze, sequenze duplicate o nucleotidi non standard che possono degradare l'apprendimento della rete neurale o generare errori a runtime. Per ovviare a questo, abbiamo implementato una robusta pipeline di preprocessing modulare (`ortho_preparation.py` e la suite di *transforms*), articolata nelle seguenti fasi:

1. **Risoluzione delle Ambiguità** (`ReplaceCharacters` / `ResolveAmbiguities`): I file FASTA grezzi sono stati scansionati per individuare caratteri anomali (es. il carattere 'N' che indica un nucleotide sconosciuto). Tramite dizionari di mapping, questi caratteri sono stati standardizzati o sostituiti per mantenere l'alfabeto strettamente aderente allo spazio delle osservazioni dell'agente.
2. **Rimozione dei Duplicati** (`RemoveDuplicates` / `ReportDuplicates`): La presenza di sequenze identiche all'interno dello stesso Orthologous Group è stata rilevata e neutralizzata. Mantenere sequenze duplicate non apporta nuova informazione biologica e rischia di generare bias nell'addestramento, facilitando artificialmente l'ottenimento di alti punteggi di Column Score.
3. **Segmentazione e Normalizzazione** (`RandomCutSequence`): Poiché le sequenze CDS originali possiedono lunghezze variabili (spesso migliaia di paia di basi), incompatibili con le dimensioni fisse della finestra di osservazione della rete neurale, è stato implementato un meccanismo di taglio randomico. Attraverso parametri di soglia (K) e intervallo (H), le sequenze vengono troncate salvaguardando l'allineamento intrinseco, generando "frammenti" gestibili.

3.4 Data Augmentation e Vettorizzazione (HDF5)

L'ultimo step della pipeline di Data Engineering (`cut_boards.py`) funge da ponte tra il dato biologico pulito e l'Environment di addestramento PyTorch. Affinché l'agente impari ad *allineare*, deve partire da uno stato *disallineato*. Utilizzando operatori stocastici custom come `RandomGapInsertion` e `RandomGapSubstitution`, introduciamo percentuali controllate di gap e "rumore" all'interno dei blocchi di sequenze perfette, creando così lo "Stato 0" (il problema da risolvere).

Infine, le stringhe nucleotidiche vengono mappate in tensori numerici interscambiabili (es. A=1, T=2, C=3, G=4, Gap=5), paddate per allinearle alla larghezza operativa W (`LeftGapPad`), e impacchettate insieme in matrici di dimensione $N \times W$ (es. 3×30). L'intero dataset processato viene salvato in un formato binario ad alte prestazioni (**HDF5**). Questa scelta architetturale è cruciale: permette al `DataLoader` di PyTorch di accedere ai batch di addestramento con latenze nell'ordine dei microsecondi, evitando il collo di bottiglia dell'I/O su disco durante le fasi intensive di training in GPU.

3.5 Impatto del Nuovo Dataset sulle Prestazioni della Baseline

Per quantificare l'effettivo valore della nuova pipeline di Data Engineering, abbiamo sottoposto le architetture di baseline (DPAMSA e GA-DPAMSA) a un test comparativo. Abbiamo valutato i modelli addestrati sul vecchio dataset sintetico contro le stesse architetture addestrate sul nuovo dataset biologico OrthoDB. Il test è stato condotto fornendo a tutti i modelli sequenze biologiche reali mai viste prima. I risultati, riassunti nella Tabella 1, mostrano un divario prestazionale netto.

Modello (Dati di Training)	Mean SP	Mean CS
DPAMSA (Sintetico)	-316.80	0.069
DPAMSA (OrthoDB)	12.80	0.344
GA-DPAMSA (Sintetico)	-82.48	0.215
GA-DPAMSA (OrthoDB)	22.40	0.359

Table 1: Confronto delle metriche di Sum of Pairs (SP) e Column Score (CS) per le architetture baseline addestrate con dati sintetici vs dati biologici (OrthoDB).

I modelli addestrati su dati casuali falliscono nell'allineare sequenze ortologhe, ottenendo punteggi di Sum of Pairs fortemente negativi. Questo conferma l'ipotesi iniziale: l'agente originale non aveva appreso i pattern evolutivi, ma si limitava a interpolare rumore stocastico. L'addestramento sui dati OrthoDB, al contrario, ha permesso alle reti di stabilizzarsi su valori positivi di SP e di quintuplicare il Column Score, dimostrando una reale capacità di generalizzazione sul dominio biologico.

3.6 Considerazioni Finali e Pipeline di Inferenza

Alla luce dell’evidente superiorità dei dati estratti da OrthoDB nel formare agenti generalizzabili, abbiamo stabilito che l’intera fase di benchmarking e valutazione delle nuove architetture (come DCNNMSA) si sarebbe svolta esclusivamente su questi dati.

Tuttavia, testare un modello su sequenze provenienti dalla stessa distribuzione solleva il rischio del *data leakage*: il modello potrebbe imparare a memoria le sequenze viste in fase di training, invalidando i test di accuratezza. Per escludere categoricamente questa eventualità, abbiamo ingegnerizzato una pipeline di inferenza isolata. Tramite uno script dedicato, un set di sequenze raw viene prelevato da una directory segregata (`inference_raw`), sottoposto alla medesima catena di preprocessing (risoluzione ambiguità, rimozione duplicati e segmentazione stocastica tramite `RandomCutSequence`), e salvato in formato FASTA in una directory di destinazione (`inference_benchmark_ready`).

Questo approccio garantisce l’assoluta impermeabilità tra i dati di addestramento (compilati in HDF5) e i dati di test (estratti a runtime come FASTA), assicurando che i risultati ottenuti in fase di benchmark riflettano la reale capacità dell’agente di allineare sequenze biologiche mai osservate in precedenza (zero-shot generalization).

4 Architetture Sperimentate

4.1 Dilated CNN-based Architectures

4.1.1 Introduzione e Motivazioni

L'architettura presentata in questa sezione nasce dalla necessità di snellire l'architettura del sistema attuale, attaccando i due principali colli di bottiglia:

- **Attention-Based Encoder:** il sistema attuale basa il meccanismo di allineamento sull'assunzione che sia possibile utilizzare sistemi di NLP per catturare caratteristiche semantiche tra nucleotidi a lungo raggio. Per farlo si utilizza un modulo basato su *attention*, che però scala in modo quadratico rispetto alla lunghezza delle sequenze;
- **Exponential Action Space:** il sistema attuale è costituito da un agente che agisce sulla singola colonna di allineamento, decidendo per ogni riga se inserire o meno un *gap*. L'architettura attuale però non gestisce in modo efficiente il fatto che lo spazio delle azioni possibili cresca in modo esponenziale nel numero di sequenze;

Il primo punto è stato gestito a partire dalla discussione dell'ipotesi: sebbene l'approccio NLP sia stato motivato e discusso in modo estensivo, gli strumenti utilizzati non permettono di addestrare un modello in tempi accettabili su un numero di dati sufficientemente grande a meno di avere a disposizione della potenza di calcolo normalmente non accessibile. L'architettura che verrà presentata successivamente si basa su un'ipotesi diversa: la maggior parte delle relazioni intra-nucleotidi si trova a piccole distanze, nell'ordine delle decine (11). Questa assunzione legittima l'uso di modelli basati su convoluzione per l'apprendimento di feature locali.

Il secondo punto è un problema noto nel panorama della progettazione di agenti di RL, e verrà gestito tramite l'adozione di una strategia action-branching, che rende la scelta dell'azione su ogni riga indipendente dalle altre, permettendo di far scalare l'architettura in modo lineare nel numero di sequenze.

Di seguito, verranno prima descritti i fondamenti teorici delle convoluzioni dilatate e dell'action branching, per poi descrivere due architetture: l'architettura basata su convoluzioni dilatate (DCNN) e l'evoluzione di questa che implementa action branching (ABDC).

4.1.2 Dilated Convolutions

Le *dilated convolutions* sono state presentate da [25] come una generalizzazione delle convoluzioni semplici dove il filtro viene progressivamente *dilatato*. La dilatazione in questione preserva la grandezza del filtro, e quindi il numero di celle, ma le dispone su un'area più grande, permettendo di fare convoluzione di informazioni progressivamente più distanti. In particolare viene definito *receptive field*(RF), la regione del dato su cui si sta facendo convoluzione che influenza il risultato (Fig. 1).

Formalmente, una convoluzione classica è definita come segue: Sia $F : \mathbb{Z}^2 \rightarrow \mathbb{R}$ una funzione discreta e $k : \Omega_r \rightarrow \mathbb{R}$ un filtro discreto di grandezza $(2r + 1)^2$ con dominio $\Omega_r = [-r, r]^2 \cap \mathbb{Z}^2$. L'operatore di convoluzione discreta $(*)$ è definito come:

$$(F * k)(p) = \sum_{s+t=p} F(s)k(t) \quad (4)$$

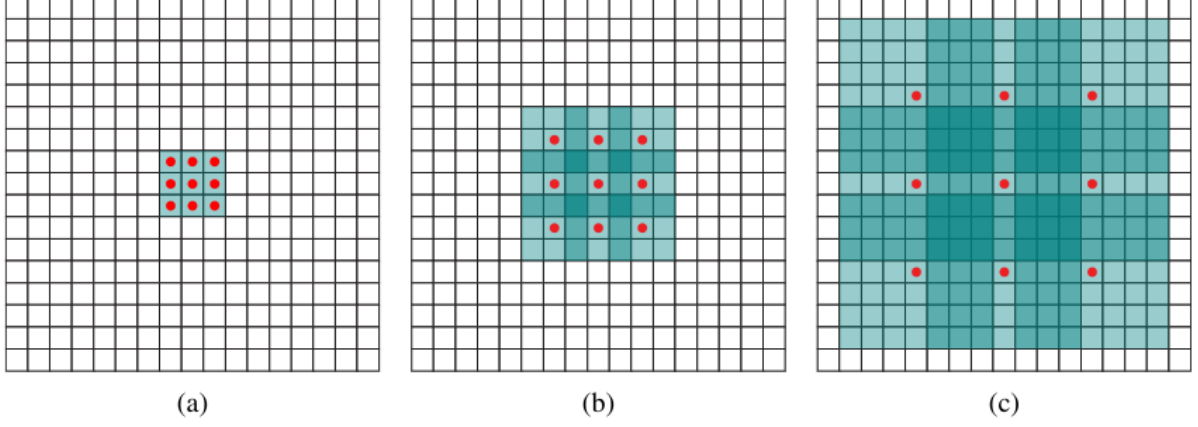


Figure 1: Dilatazione progressiva del receptive field. I punti rossi sono le celle il cui valore è calcolato come convoluzione delle celle colorate. L'intorno di ogni punto rosso è progressivamente più grande. In questo esempio in (a) ogni elemento ha un receptive field di 3×3 , (b) 7×7 e (c) 15×15 . Mentre il receptive field cresce in modo esponenziale, il numero di parametri cresce in modo lineare con il numero di convoluzioni.

Sia l un fattore di dilatazione, si generalizza la convoluzione discreta con il simbolo $(*_l)$ (convoluzione l -dilatata) come segue:

$$(F *_l k)(p) = \sum_{s+lt=p} F(s)k(t) \quad (5)$$

Grazie a questa formalizzazione possiamo descrivere la convoluzione l -dilatata come un caso più generale della convoluzione discreta.

Questa architettura supporta dilatazioni progressive che dilatano il filtro in modo esponenzialmente grande. Si immagini di avere le funzioni discrete $F_0, F_1, \dots, F_{n-1} : \mathbb{Z}^2 \rightarrow \mathbb{R}$ e $k_0, k_1, \dots, k_{n-2} : \Omega_1 \rightarrow \mathbb{R}$ filtri discreti. Si può costruire un'architettura che computa:

$$F_{i+1} = F_i *_2 k_i \quad \text{per } i = 0, 1, \dots, n-2 \quad (6)$$

dove il RF di F_{i+1} è una matrice di grandezza $(2^{i+2} - 1) \times (2^{i+2} - 1)$.

4.1.3 Double Dueling Q-Network

L'architettura Double Dueling Q-Network (DDQN) [24] è un'architettura pensata per migliorare la stima della policy negli algoritmi di RL model free stimando separatamente *state-value* ed *advantage* invece che stimare direttamente i *Q-values*. Formalmente, dato lo stato s e l'azione a , vale la relazione

$$Q(s, a) = V(s) + A(s, a) \quad (7)$$

In questa relazione sono stati omessi i parametri della rete che li stima per brevità. Da 7 si ha che sebbene la relazione tra action-value e advantage sia evidente, il q-value non è univocamente identificato da una coppia di queste. Tanti valori di action-value e advantage possono essere aggregati nello stesso q-value, che è un problema per l'apprendimento dell'agente. Questo problema viene aggirato imponendo che lo stimatore dell'advantage function valga zero sulla migliore azione. Si deriva quindi:

$$Q(s, a) = V(s) + \left(A(s, a) - \max_{a' \in |A|} A(s, a') \right) \quad (8)$$

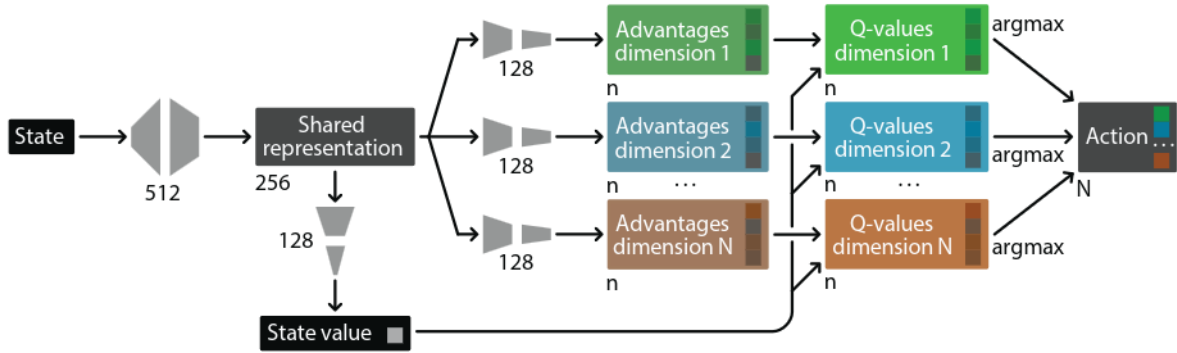


Figure 2: Esempio di architettura Branching Dueling Deep Q Network (BDQ)

4.1.4 Action Branching

L'architettura action branching nasce per mitigare il problema di agenti di RL con spazio di azioni esponenzialmente grande[20]. L'idea è quella di predisporre un modulo di *shared representation* (SR) che funge da base di conoscenza per coordinare una serie di *branches* che indipendentemente fanno scelte atomiche. Unendo gli output delle ramificazioni si ottiene l'azione totale dell'agente. Il principio dell'action branching viene implementato con l'architettura Branching Dueling Deep Q-Network (BDQ) (Fig. 2). Questa architettura attua il meccanismo del branching solo su degli stimatori di advantage, lasciando la stima dello state-value ad una testa separata. Ogni testa di advantage viene poi aggregata alla testa di state, producendo una stima del q-value per ogni azione atomica. Infine queste vengono aggregate in un unico modulo decisionale.

4.1.5 Reward system

Il sistema di premi e penalità differisce nelle due architetture principalmente per i premi e le penalità utilizzate. Tuttavia l'idea sottostante è la medesima: data una colonna a seguito dell'azione dell'agente, esso riceve un premio per ogni coppia di nucleotidi uguali (match) e una penalità per ogni coppia di nucleotidi diversi (mismatch). Per quanto riguarda i gap sono state valutate due strategie: la prima considerava il gap come una sorta di *meta-nucleotide* che potesse solo fare mismatch con gli altri nucleotidi. Questa strategia è stata considerata troppo punitiva. Si pensi per esempio alla colonna allineata come segue $[A, A, -]$ (dove il trattino indica un gap inserito). Secondo la strategia presa in considerazione c'è una coppia di nucleotidi che fa match e due coppie nucleotide-gap. Sebbene questo sia un allineamento parziale probabilmente vantaggioso, l'agente riceve un premio e due penalità. Invece che modificare i pesi, si è optato per una strategia migliore: la penalità per l'aggiunta di gap è moltiplicata per il numero di gap inseriti, non trattando più il gap come un nucleotide. In questo modo si evitano anche problemi di aumento esponenziale delle penalità per allineamenti più grandi. Infine, la scelta precisa dei valori di premi e penalità è stata fatta prima su uno studio combinatorio, per calcolare il premio cumulativo medio in caso di azioni casuali e la frequenza delle penalità, e poi tramite aggiustamenti empirici.

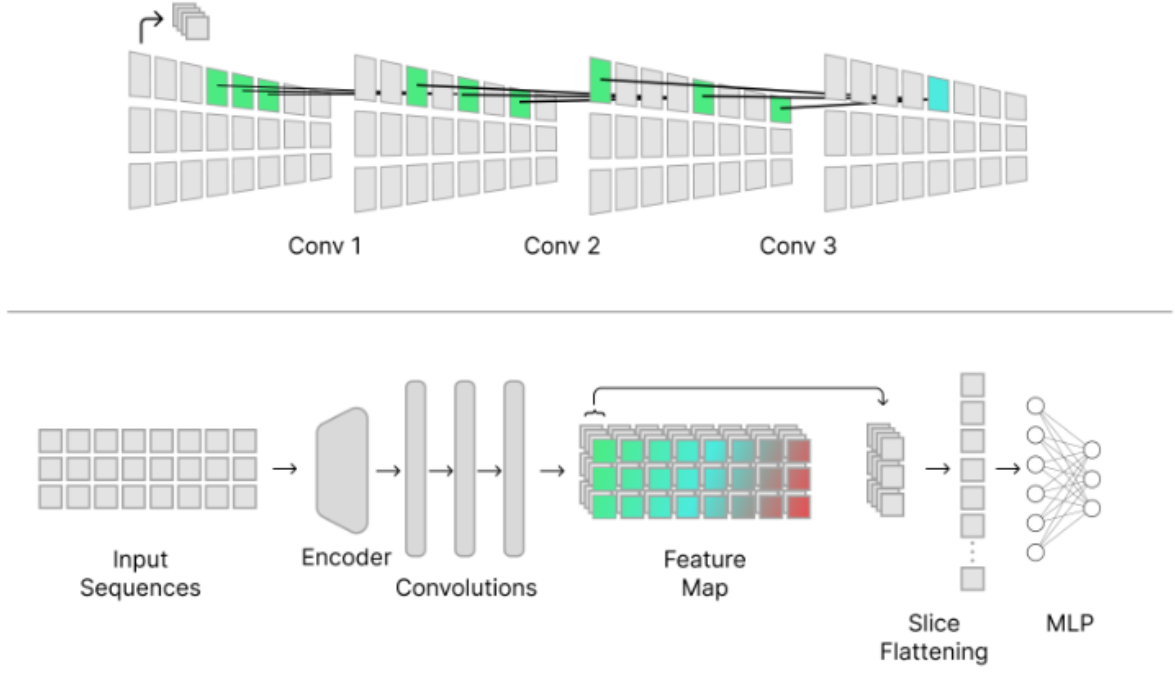


Figure 3: Architettura DCNN e illustrazione delle convoluzioni dilatate

4.1.6 Dilated Convolutions-based architecture

L'architettura presentata attua un'elaborazione stepwise, analizzando tutta la finestra di allineamento ma prendendo una decisione solo sulla prima colonna. Questa verrà poi eliminata e al modello verrà presentata la finestra restante, iterando su di essa fino a consumarla completamente. Come descritto in Fig. 3, la matrice della finestra di allineamento viene data in input ad un modulo di encoding, che codifica in una rappresentazione vettoriale densa ogni nucleotide per evitare bias di codifica numerica. Vengono effettuate tre convoluzioni dilatate con dilatazione esponenziale, coprendo un receptive field di 15 nucleotidi. Per questa architettura, i filtri convoluzionali sono stati pensati come orizzontali per evitare che il filtro imparasse relazioni d'ordine tra le sequenze. Questa scelta verrà considerata poco praticabile nella prossima architettura. Da questa osservazione possiamo apprendere che non è la fase di convoluzione a cogliere le relazioni inter-sequenze, ma serve a catturare caratteristiche di ordinamento e pattern intra-sequenze. Viene quindi estratta la prima colonna, appiattita e poi data come input ad un MLP. In questo momento il modello fonde la conoscenza acquisita nel modulo precedente per inferire delle relazioni tra i nucleotidi di diverse sequenze. L'output del MLP è un vettore binario di dimensione 2^n , con n numero di sequenze nell'allineamento.

Addestramento e iperparametri Il modello è stato addestrato su un dataset di 20.000 allineamenti di sequenze ortologhe, ogniuna considerata come una epoca di training. Ogni allineamento è composto da 3 sequenze di lunghezza 30. Vista la bassa sparsità dei dati, è stato considerato sufficiente uno split 95% training set e 5% test set. È stato incluso un replay buffer di grandezza 1000 e un refresh rate della policy network di 1000. Il discount factor è stato calcolato usando la seguente euristica:

$$\#steps = \frac{1}{1 - \gamma}$$

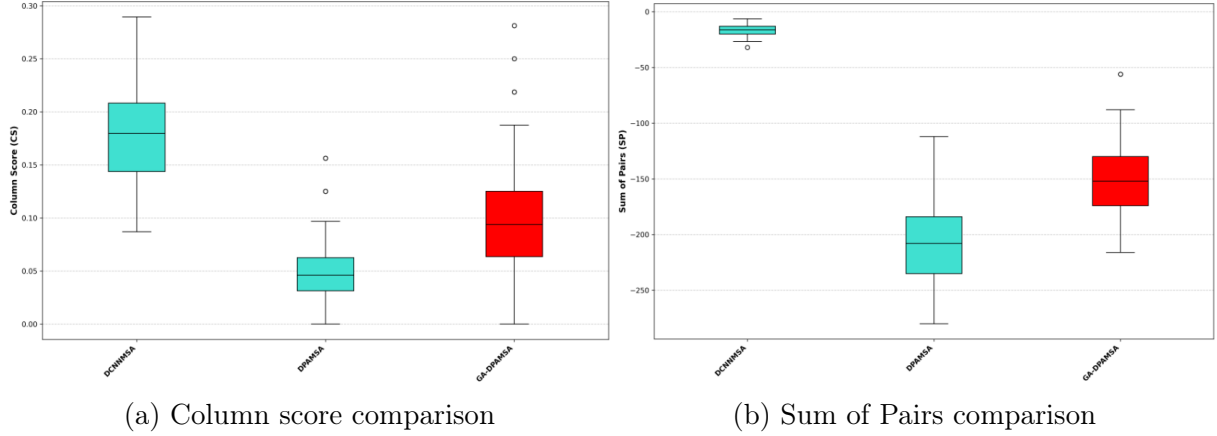


Figure 4: Comparazione dei modelli DCNN (sinistra), DPAMSA (centro) e GA-DPAMSA (destra) per metriche di CS (a) e SP (b). A sinistra il modello proposto, in rosso il miglior modello precedente.

Ovvero è stato trovato in modo che il numero di passi che influenzassero il calcolo del q-value attuale ($\#steps$) fosse proporzionato alla grandezza dell'allineamento. Il valore trovato è $\gamma = 0.9$. È stato impostato un valore di esplorazione $\varepsilon = 0.95$ con decadimento esponenziale di 0.9998, per ottimizzare il tradeoff di *exploration-exploitation*. Il learning rate $\alpha = 1e - 5$ è stato trovato empiricamente valutando la stabilità del training. Infine, premi e penalità sono stati lasciati come per il sistema DPAMSA, quindi *match reward* a +4, e *gap penalty* e *mismatch penalty* a -4.

Sperimentazione e Risultati Il modello è stato testato su 1000 sequenze ortologhe e comparato ai sistemi precedenti (DPAMSA e GA-DPAMSA). Il modello supera in entrambe le metriche il modello precedentemente proposto (Fig. 4).

4.1.7 Action Branching and DC-based architecture

L'architettura precedente soffre ancora di problemi di scalabilità sullo spazio delle azioni. Per risolverlo è stato integrato un modulo di action branching, assieme ad altre modifiche per stabilizzare l'addestramento. Una prima differenza è nella forma del filtro convoluzionale, che questa volta è progettato per coprire tutte le sequenze in altezza. Questa scelta è stata fatta per evitare di dover disporre di un modulo di aggregazione delle informazioni inter-sequenza in vista di test su allineamenti con più di tre sequenze. Come prima, viene estratta la prima colonna della feature map e appiattita. Viene poi effettuata una compressione tramite un MLP apposito, e l'output diventa l'input di due moduli: quello di stima della *state-value* e ad una serie di n moduli (per n sequenze da allineare) di stima della *advantage*, ognuno responsabile dell'inserimento o meno di un *gap*. Gli output delle teste di advantage e di value sono stati aggregati tramite media degli advantage come spiegato nel paper originale [24].

Addestramento e iperparametri Non sono state effettuate importanti modifiche agli iperparametri e alle modalità di addestramento della rete. L'unica differenza sta nella strategia di decadimento dell'exploration rate, impostata come lineare, e nella scelta dei rewards. Per questi ultimi, è stato necessario bilanciare meglio i pesi per assicurare la convergenza dell'agente. *match reward* è stato impostato a +6, *mismatch penalty* a -3 e *gap*

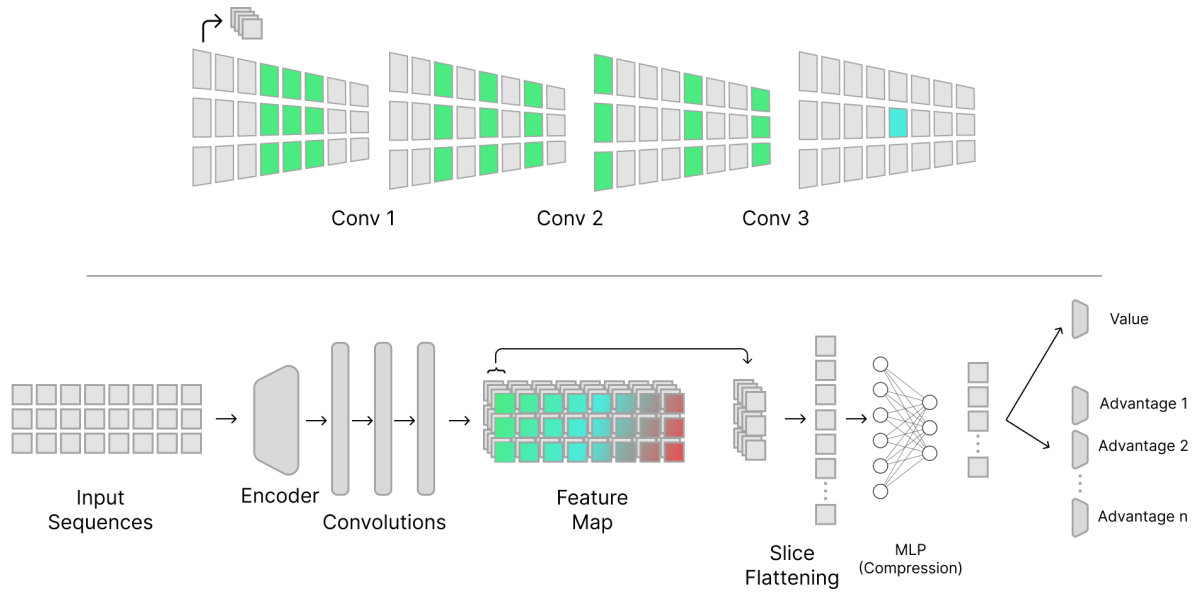


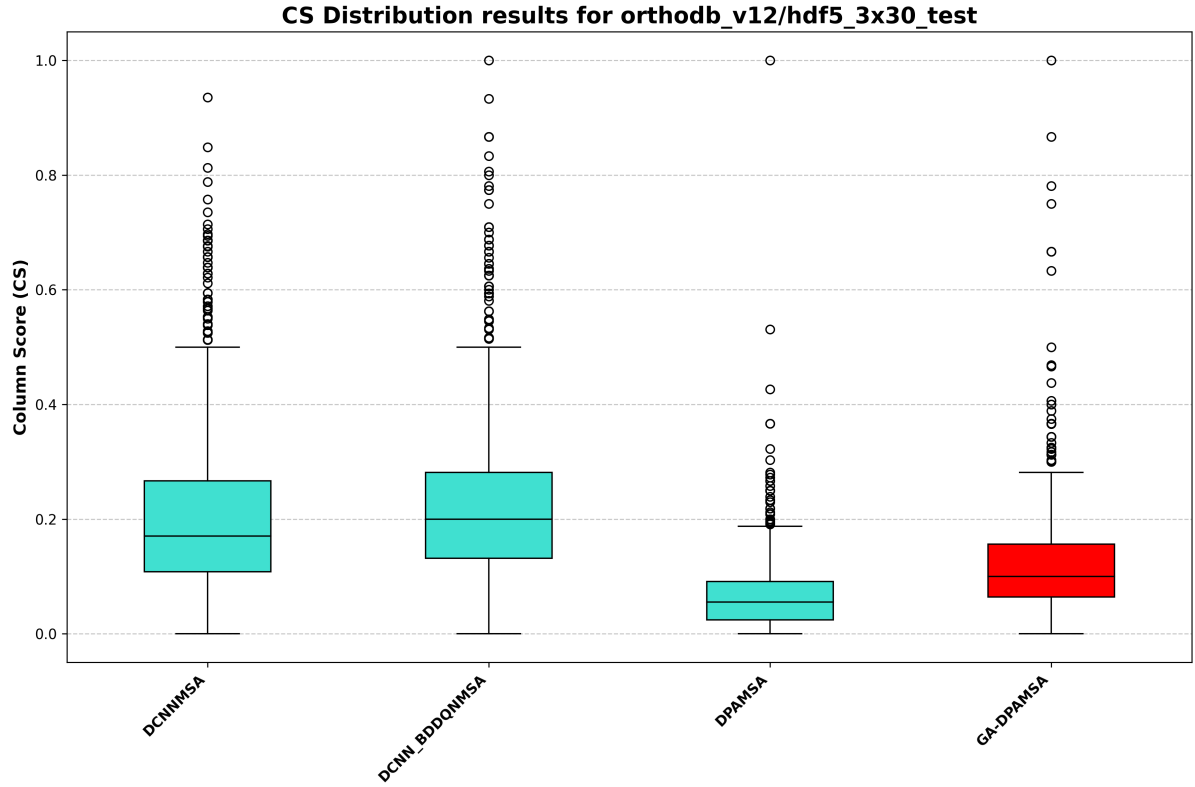
Figure 5: Estensione di architettura DCNN con Action Branching e illustrazione delle convoluzioni dilatate

penalty a -2 . Inoltre è stato necessario definire una penalità costante sul tempo per evitare che l'agente imparasse azioni che non avanzavano lo stato dell'allineamento, ovvero l'aggiunta di una colonna di soli gap. Quest ultima necessità nasce dal fatto che mentre l'architettura precedente poteva escludere in modo strutturale le azioni dannose, questa deve necessariamente includerle tutte. L'agente deve quindi imparare che alcune azioni sono particolarmente dannose, come appunto l'azione di riempire la colonna attuale di gaps.

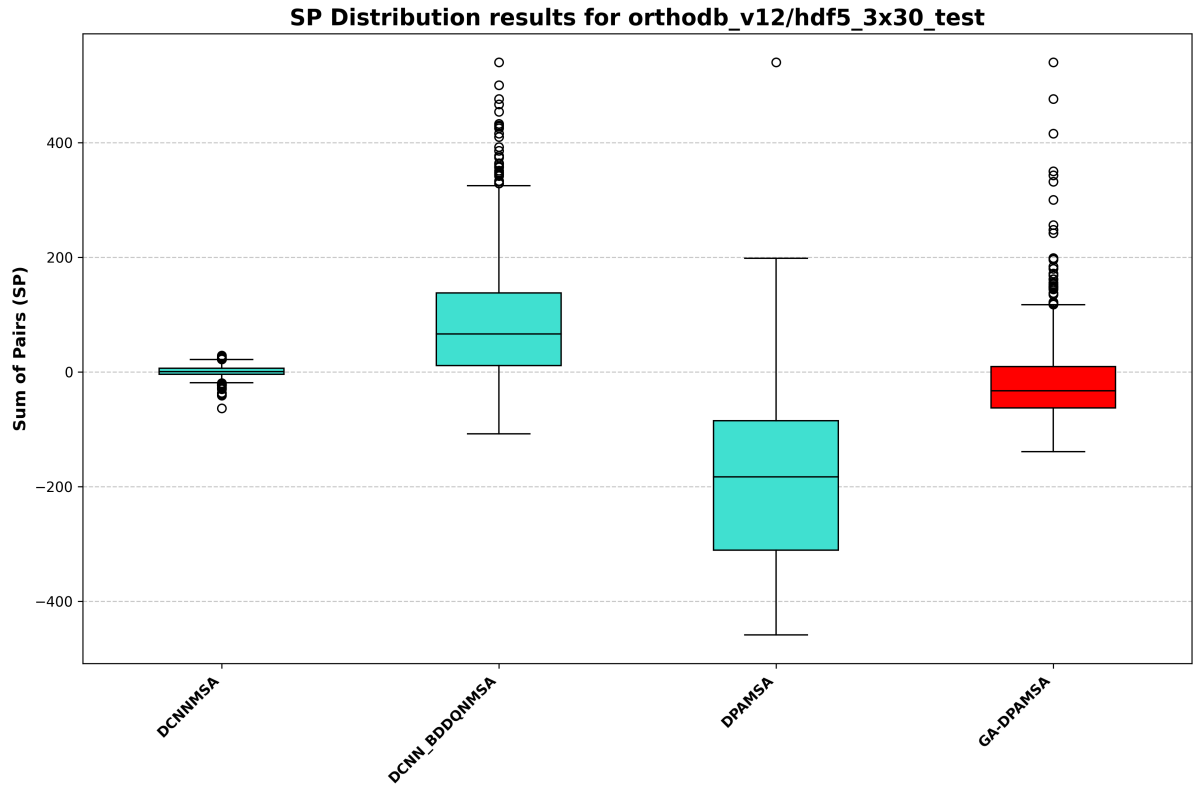
Sperimentazione e Risultati Il modello è stato testato sotto le stesse condizioni dell'architettura precedente, confermando e aumentando il distacco con i modelli baseline.

4.1.8 Discussione e lavori futuri

I due modelli presentati migliorano le prestazioni sul task di allineamento rispetto ai modelli precedentemente proposti. Essi, infatti, possono essere addestrati in modo molto più rapido su una mole di dati molto maggiore e sono complessivamente più leggeri grazie al fatto che i filtri convoluzionali hanno grandezza costante e al fatto che non si fa uso di un modulo di *attention* a differenza di come proposto sui modelli baseline. In particolare, l'architettura che combina approccio di convoluzioni dilatate e action branching risulta essere teoricamente più scalabile. Al momento esperimenti di successo con questa architettura su allineamenti con più sequenze non sono ancora riusciti. Tuttavia, riteniamo che il problema sia principalmente combinatorio: la probabilità che ci sia un match tra due nucleotidi in un allineamento casuale diventa certa una volta che si comincia a lavorare su allineamenti da 5 sequenze in su. Come osservato da ulteriori esperimenti, l'agente riceve quindi tanti premi per azioni che casualmente hanno portato a non modificare i nucleotidi già allineati, portandolo a convergere verso una policy errata. Riteniamo quindi che il problema non sia architetturale, ma che l'approccio sia valido a patto di riuscire a pro-



(a) Distribuzione Column Score



(b) Distribuzione Sum of Pairs

Figure 6: Comparazione CS (a) e SP (b) tra architettura basata unicamente su convoluzioni dilate (DCNNMSA), con aggiunta di action branching (DCNN-BDDQNMSA), e modelli baseline DPAMSA e GA-DPAMSA. In rosso il miglior modello baseline.

gettare una reward function che consideri anche questo caso. Un esempio potrebbe essere quello di non utilizzare un reward assoluto, ma di erogare un premio relativo proporzionale a quanto la colonna in allineamento sia migliorata a seguito dell'azione dell'agente. Attualmente esperimenti di questo tipo sono ancora in corso e costituiscono buona parte del lavoro che potrà essere fatto in futuro. Un altro importante aspetto che non è ancora stato valutato è se un agente di RL possa apprendere un'euristica di allineamento che possa avere senso biologico, facendo emergere pattern evolutivi attraverso l'allineamento progressivo.

4.2 Allineamento One-Shot con PPO e GRPO

In parallelo agli approcci stepwise descritti nelle sezioni precedenti, abbiamo esplorato una direzione alternativa basata sull'allineamento one-shot, in cui il modello produce l'intero allineamento in un'unica inferenza. Questa formulazione elimina la propagazione sequenziale degli errori e riduce l'inferenza a un singolo step, con un vantaggio significativo in termini di velocità. Sul piano algoritmico, abbiamo adottato Proximal Policy Optimization (PPO) [17] e Group Relative Policy Optimization (GRPO) [18] in sostituzione del Deep Q-Learning di DPAMSA: si tratta di approcci più recenti, basati sull'ottimizzazione diretta della policy e nativamente parallelizzabili.

4.2.1 Proximal Policy Optimization

PPO [17] è un algoritmo di reinforcement learning on-policy che ottimizza la policy attraverso un obiettivo surrogato con clipping. L'idea centrale è limitare l'entità degli aggiornamenti ad ogni iterazione, evitando cambiamenti troppo bruschi che potrebbero destabilizzare l'addestramento. Questo vincolo rende inoltre l'algoritmo naturalmente parallelizzabile: poiché l'aggiornamento è confinato in un intorno controllato della policy precedente, è possibile raccogliere esperienze in parallelo senza rischi di instabilità. L'obiettivo ottimizzato è:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (9)$$

dove $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ è il rapporto tra la policy aggiornata e quella precedente, $\hat{A}_t$ è la stima del vantaggio, e ϵ è l'iperparametro che controlla l'ampiezza massima dell'aggiornamento. La stima del vantaggio viene calcolata attraverso una rete critic separata, che apprende a predire il valore atteso del ritorno a partire dallo stato corrente.

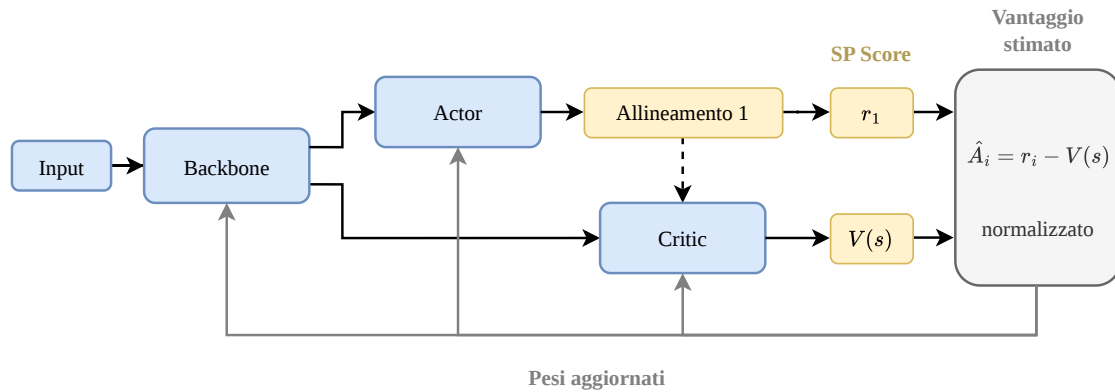


Figure 7: Architettura basata su PPO: la rete Actor genera l'allineamento, mentre il Critic separato stima la funzione valore.

4.2.2 Group Relative Policy Optimization

GRPO [18] condivide con PPO la stessa filosofia di aggiornamento conservativo della policy, ma si distingue nel meccanismo di stima del vantaggio. Aniché utilizzare una rete critic dedicata, GRPO genera per ogni input un gruppo di G output campionati dalla

policy corrente, ciascuno dei quali riceve un reward. Il vantaggio di ciascun campione viene calcolato in termini relativi rispetto al gruppo:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G} \quad (10)$$

dove r_i è il reward dell' i -esimo campione e μ_G , σ_G sono rispettivamente media e deviazione standard dei reward del gruppo. Questa formulazione elimina la necessità di addestrare un critic, riducendo la complessità del modello e il costo computazionale. Nel contesto dell'allineamento one-shot, il meccanismo risulta naturale: ogni campionamento produce un allineamento completo, fornendo un segnale di apprendimento robusto senza stimare esplicitamente una funzione valore.

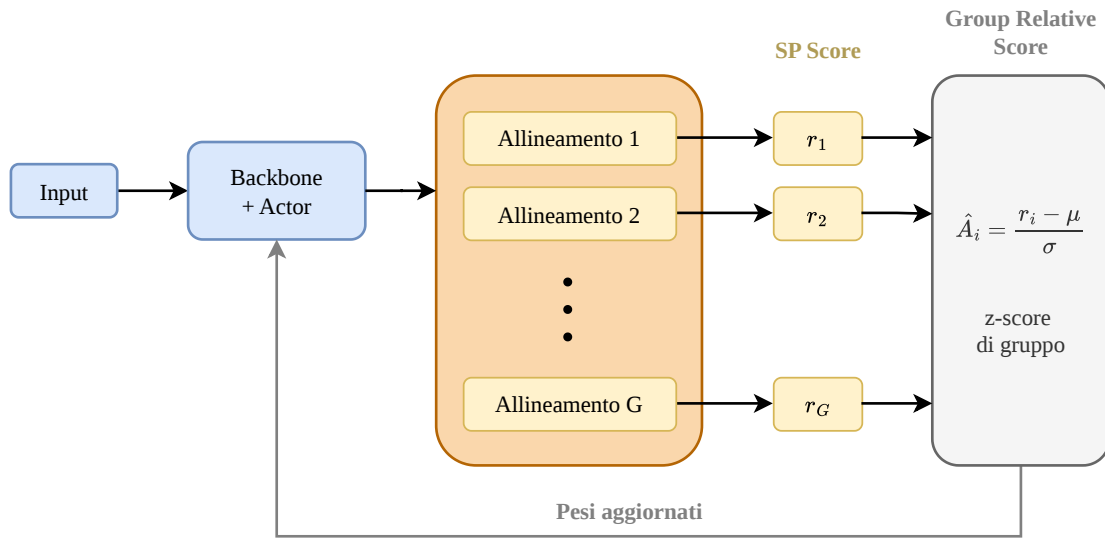


Figure 8: Architettura basata su GRPO: calcolo del vantaggio relativo su un gruppo di campioni, senza l'uso di una rete Critic.

4.2.3 Componenti utilizzati

Poiché PPO e GRPO condividono la stessa struttura di base e differiscono esclusivamente nel meccanismo di stima del vantaggio, l'implementazione è stata organizzata in modo modulare attorno a componenti riutilizzabili da entrambi i paradigmi:

- **Backbone:** rete convoluzionale dilatata [25] con embedding e codifica posizionale sinusoidale, già introdotta e descritta precedentemente, che elabora la matrice di allineamento producendo una rappresentazione latente utilizzata dalle teste successive.
- **Actor:** MLP a tre layer con normalizzazione, responsabile della generazione delle azioni di allineamento.
- **Critic:** MLP a due layer con normalizzazione, utilizzato esclusivamente nella variante PPO per la stima della funzione valore.

4.2.4 Environment vettorizzato

Il passaggio a una formulazione one-shot ha reso necessaria la progettazione di un environment dedicato, non compatibile con quello stepwise utilizzato da DPAMSA. In un contesto one-shot, l'environment riceve in un'unica chiamata l'intera azione dell'agente (ovvero la distribuzione dei gap per ogni posizione di ogni sequenza) e deve restituire l'allineamento risultante insieme al relativo reward.

Per sfruttare appieno la parallelizzabilità degli algoritmi di addestramento, l'environment è stato implementato in modo interamente vettorizzato su GPU tramite operazioni PyTorch. La ricostruzione dell'allineamento avviene attraverso un meccanismo di *scatter*, che calcola le posizioni finali dei nucleotidi e costruisce l'intera matrice in un singolo passaggio tensoriale, senza loop espliciti.

Per la stessa ragione, anche le funzioni di scoring (Sum of Pairs e Column Score) sono state reimplementate in PyTorch con operazioni vettorizzate, evitando che il calcolo del reward diventasse un collo di bottiglia rispetto al resto della pipeline. L'intera catena, dall'azione dell'agente al reward, opera quindi su GPU senza trasferimenti intermedi verso CPU.

4.2.5 Interpretazione dell'Output

Un aspetto distintivo di questa architettura è la formulazione dell'output come distribuzione continua. L'output del modello viene trasformato in un allineamento concreto attraverso tre fasi successive: parametrizzazione della distribuzione, campionamento e decodifica, e ricostruzione tramite scatter vettorizzato.

Parametrizzazione. Per ogni posizione (i, j) della matrice di input (riga i , colonna j), la rete produce un vettore di logit bidimensionale. Il primo componente parametrizza la media μ della distribuzione, il secondo la log-deviazione standard $\log \sigma$:

$$\mu_{i,j} = \text{Softplus}(z_{i,j,0}), \quad \sigma_{i,j} = \exp\left(\text{clamp}(z_{i,j,1}, \log \sigma_{\min}, \log \sigma_{\max})\right) \quad (11)$$

dove $\log \sigma_{\min} = -2.0$ e $\log \sigma_{\max} = 2.0$. L'uso della Softplus per μ garantisce valori strettamente positivi, coerenti con l'interpretazione di conteggio di gap. Le posizioni corrispondenti a padding vengono mascherate forzando $\mu = 0$ e $\sigma \approx 0$, impedendo alla rete di apprendere segnali spuri al di fuori delle sequenze reali. L'output complessivo ha quindi dimensione $B \times N \times L \times 2$, dove B è il batch size, N il numero di sequenze e L la lunghezza della finestra operativa.

Campionamento e decodifica. In fase di addestramento, il conteggio di gap per ogni posizione viene campionato dalla distribuzione normale parametrizzata:

$$a_{i,j} \sim \mathcal{N}(\mu_{i,j}, \sigma_{i,j}^2) \quad (12)$$

Il valore campionato viene poi rettificato e discretizzato:

$$g_{i,j} = \min\left(\lfloor f(a_{i,j}) + 0.5 \rfloor, g_{\max}\right) \quad (13)$$

dove f è la funzione di rettifica (ReLU per PPO, Softplus per GRPO) e $g_{\max} = 5$ è il numero massimo di gap consentiti per posizione. Il valore $g_{i,j}$ rappresenta il numero di gap da inserire *prima* del nucleotide in posizione j della sequenza i . In fase di inferenza, si utilizza direttamente la media μ al posto del campionamento stocastico, producendo un allineamento deterministico.

Ricostruzione tramite Scatter. La conversione da conteggi di gap ad allineamento concreto avviene interamente su GPU tramite un meccanismo di *scatter* vettorizzato, senza loop espliciti. Per ogni sequenza, si calcola lo shift cumulativo dei gap:

$$s_{i,j} = \sum_{k=1}^j g_{i,k} \quad (14)$$

La posizione finale del nucleotide originariamente in colonna j diventa $j + s_{i,j}$. Si inizializza una matrice canvas di dimensione $N \times L'$ riempita con token di gap (dove $L' = \max_{i,j}(j + s_{i,j}) + 1$), e si proiettano i nucleotidi originali nelle posizioni calcolate tramite l'operazione `scatter_`. Il padding dinamico di L' evita sprechi di memoria rispetto a una lunghezza fissa prestabilita.

4.2.6 Addestramento e Iperparametri

La configurazione di addestramento è stata progettata per essere il più possibile uniforme tra i due algoritmi, in modo da isolare l'effetto della diversa stima del vantaggio come unica variabile sperimentale. La Tabella 2 riassume i parametri utilizzati nelle run di training finali.

Table 2: Configurazione di addestramento per le architetture PPO e GRPO.

Parametro	PPO	GRPO
Ottimizzatore	Adam	Adam
Learning rate (α)	1×10^{-4}	1×10^{-4}
Batch size	64	64
Epoche totali	100	100
Epoche interne per batch (K)	4	4
Clip epsilon (ε)	0.2	0.2
Coefficiente entropia (c_{ent})	0.005	0.005
Coefficiente value loss (c_v)	0.5	—
Gradient clipping (max norm)	1.0	1.0
Group size (G)	—	32
Segnale di reward	SP	SP
Rettifica azioni	ReLU	Softplus

La formulazione one-shot riduce l'episodio a un singolo step decisionale, rendendo il ritorno coincidente con il reward immediato. Per questa ragione non si utilizzano né discount factor né Generalized Advantage Estimation. Il segnale di reward è costituito esclusivamente dal Sum of Pairs, calcolato in modo vettorizzato su GPU dall'environment descritto nella Sezione 4.2.4. Il Column Score viene calcolato in parallelo a fini di monitoraggio ma non partecipa all'ottimizzazione della policy.

Un dettaglio implementativo che distingue le due varianti riguarda la rettifica delle azioni continue in conteggi interi di gap: PPO applica ReLU, GRPO utilizza Softplus. Quest'ultima, essendo liscia e strettamente positiva, garantisce gradienti non nulli in prossimità dello zero, producendo un segnale di apprendimento più stabile. L'entropia viene calcolata esclusivamente sulle posizioni valide della matrice, escludendo il padding per evitare di diluire artificialmente il segnale su sequenze più corte della finestra operativa.

4.2.7 Sperimentazione e Risultati

Entrambi i modelli sono stati testati sullo stesso dataset di 1000 sequenze ortologhe utilizzato per le architetture precedenti. Come mostrato in Figura 9, entrambi gli algoritmi migliorano significativamente rispetto alle baseline, ottenendo per la prima volta valori positivi di Sum of Pairs (SP: 19.04) e più che quadruplicando il Column Score rispetto a DPAMSA (CS: 0.354 vs 0.076). I due algoritmi convergono verso prestazioni identiche su questo benchmark.

Sebbene i valori superino le baseline, l’approccio one-shot non raggiunge le prestazioni delle architetture stepwise presentate nelle sezioni precedenti. Un’analisi comparativa completa con tutti i modelli sviluppati e con i tool dello stato dell’arte è riportata nella Sezione 5.

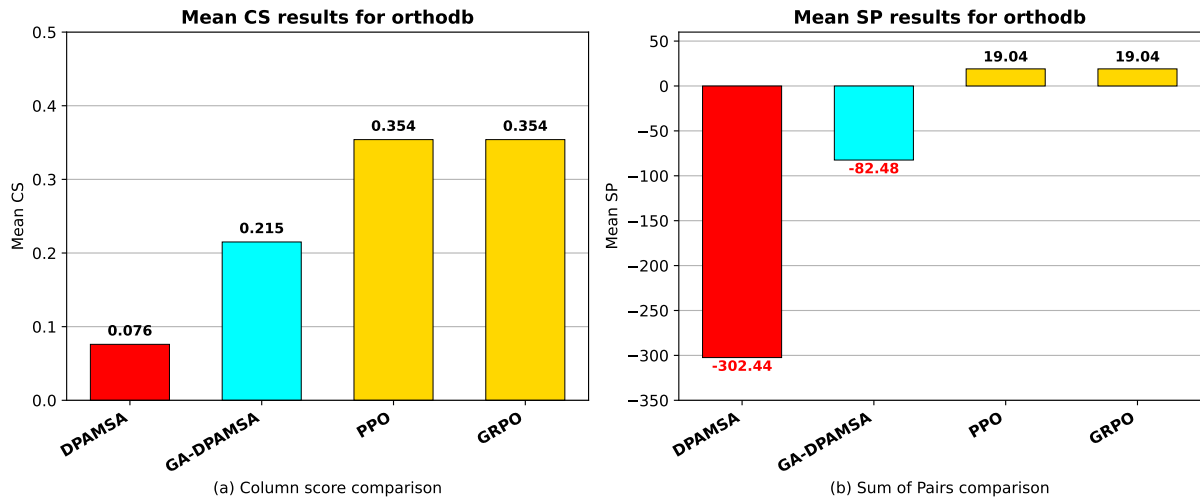


Figure 9: Comparazione dei valori medi di Column Score (a) e Sum of Pairs (b) per gli approcci one-shot (PPO, GRPO) rispetto alle baseline (DPAMSA, GA-DPAMSA). In rosso il modello base.

4.2.8 Discussione e Lavori Futuri

I risultati ottenuti mostrano che la formulazione one-shot è una direzione promettente ma ancora in fase di maturazione. L’architettura attuale non raggiunge le prestazioni degli approcci stepwise, evidenziando margini di miglioramento significativi.

PPO e GRPO convergono verso prestazioni identiche, suggerendo che nella formulazione a singolo step la differenza nel meccanismo di stima del vantaggio diventi trascurabile. Questo rende GRPO preferibile per il minor costo computazionale, ma evidenzia al contempo che il collo di bottiglia prestazionale risiede nella formulazione del problema piuttosto che nell’algoritmo di ottimizzazione.

Le principali criticità identificate e le relative direzioni di intervento sono:

- **Discretizzazione dell’output:** l’arrotondamento delle azioni continue spezza il grafo computazionale. Tecniche di *straight-through estimation* o una riformulazione categorica potrebbero mitigare il problema.
- **Assenza di feedback intermedi:** formulazioni ibride (one-shot iniziale seguito da raffinamento stepwise) potrebbero combinare la velocità di inferenza con la capacità di correzione iterativa.

- **Segnale di reward:** l'integrazione del Column Score nel reward, tramite scalarizzazione o formulazione multi-obiettivo, potrebbe migliorare la qualità biologica degli allineamenti.
- **Scalabilità e spazio delle azioni:** il comportamento su finestre più ampie di 3×30 e l'adozione di distribuzioni più espressive (miscele di gaussiane, categoriche) restano da verificare.

4.3 Sviluppo del Nuovo Orchestratore Basato su NSGA-II

Ulteriori analisi e cambiamenti rispetto alla baseline di partenza si sono attuati studiando le criticità dell'orchestratore attuale, al fine di individuare un algoritmo genetico capace di gestire meglio la complessità biologica.

4.3.1 Analisi delle criticità della Baseline

Il GA utilizzato nella versione originale di GA-DPAMSA, pur essendo funzionale, presentava limiti strutturali evidenti, specialmente su dataset con molte sequenze. Sebbene introducesse una modalità Multi-Objective (MO), la sua implementazione si limitava a calcolare separatamente il Sum-of-Pairs (SP) e il Column Score (CS), per poi unire gli indici dei migliori individui di entrambe le metriche in fase di selezione (`top_indices_sop` e `top_indices_column`). Questo approccio, basato su un'unione forzata e privo di un reale criterio di dominanza, ha fatto emergere tre criticità principali:

1. Si ha uno sbilanciamento delle soluzioni, dato che l'unione degli indici non garantisce la sopravvivenza di individui equilibrati. Al contrario, favorisce individui agli "*estremi*", ovvero con prestazioni eccellenti in un parametro ma nettamente insufficienti nell'altro.
2. In assenza di un meccanismo per preservare la varietà della popolazione, l'algoritmo tendeva a convergere prematuramente verso una singola configurazione dominante. Questo portava a scartare soluzioni alternative potenzialmente molto valide dal punto di vista biologico.
3. Il meccanismo di Hall of Fame salvava l'individuo migliore basandosi esclusivamente su un singolo parametro. Questa scelta si è rivelata inappropriata per l'allineamento genomico, che per sua natura richiede un compromesso costante tra coerenza globale (SP) e conservazione locale (CS).

Di conseguenza, l'algoritmo di baseline faticava a individuare e preservare gli elementi realmente più promettenti della popolazione, rendendo necessario il passaggio a una vera architettura multiobiettivo.

4.3.2 Ricerca Locale vs Ricerca Globale: Limiti degli Algoritmi Ibridi

Dopo aver individuato i punti critici della baseline, è stata avviata una fase di studio per la riprogettazione dell'orchestratore. Inizialmente, si è valutata l'opzione di adottare varianti genetiche ibride più complesse, come i Gradient-based Genetic Algorithms (GGA) [5]. Questi approcci integrano meccanismi di ricerca locale, spesso basati sulla discesa del gradiente, per accelerare la convergenza verso un minimo globale. Tuttavia, l'analisi del nostro specifico dominio applicativo ha portato a escludere tali metodi in favore di algoritmi genetici improntati su una ricerca puramente globale, sulla base di tre motivazioni strutturali:

- **Natura discreta dello spazio di ricerca:** I metodi ibridi basati su gradiente eccellono in problemi di ottimizzazione continua, dove le derivate guidano agilmente la soluzione verso i minimi locali. Il Multiple Sequence Alignment (MSA), al contrario, è un problema intrinsecamente discreto e combinatorio (basato sull'inserimento e lo spostamento di gap). Adattare una discesa del gradiente a uno spazio discreto

richiede approssimazioni matematiche (*continuous relaxations*) che introducono un **overhead computazionale gravoso** e spesso impreciso.

- **Dimensionalità della finestra operativa:** Nel nostro progetto, l'algoritmo agisce su sotto-regioni (*sub-board*) di dimensioni estremamente ridotte (attualmente fissate a finestre 3×30 , con previsioni di scaling futuro). Applicare un meccanismo iterativo di ricerca locale matematicamente pesante su una matrice così piccola risulta computazionalmente sproporzionato. Inoltre, nel nostro framework, la vera e propria "ricerca locale" è già delegata all'agente predittivo (la DCNN), rendendo ridondante un'ulteriore ottimizzazione locale a livello di algoritmo genetico.
- **Convergenza Monodimensionale vs Frontiera di Pareto:** Gli algoritmi ibridi complessi sono tipicamente ottimizzati per accelerare la convergenza verso un singolo minimo globale di una funzione di fitness scalare (gestendo eventuali vincoli tramite penalità). Tuttavia, come emerso dall'analisi della baseline, l'MSA necessita dell'esplorazione di un trade-off reale (SP vs CS). Gli algoritmi di ricerca globale multi-obiettivo sono nativamente progettati per la dominanza paretiana, garantendo la mappatura di diverse soluzioni biologiche alternative; un aspetto vitale che una convergenza locale forzata rischierebbe di annullare.

Per queste ragioni, si è stabilito che l'architettura ottimale dovesse prevedere un algoritmo focalizzato esclusivamente sulla ricerca globale e sulla gestione del compromesso multi-obiettivo, lasciando la risoluzione a grana fine all'efficienza della rete neurale.

4.3.3 Architettura utilizza e Motivazioni

La scelta di implementare il Non-dominated Sorting Genetic Algorithm II (NSGA-II) come orchestratore per l'allineamento di sequenze multiple di nucleotidi nasce dalla natura intrinsecamente conflittuale del problema. Nell'MSA, massimizzare il Sum-of-Pairs (SP, che valuta la qualità globale degli allineamenti a coppie) porta spesso alla frammentazione dei gap, riducendo drasticamente il Column Score (CS, che premia le colonne perfettamente allineate). A differenza della baseline che tentava un bilanciamento "forzato" delle due metriche, NSGAII affronta il problema trattando SP e CS come obiettivi ortogonali. L'algoritmo non cerca una singola soluzione ottimale, ma evolve una popolazione verso il **Fronte di Pareto**, ovvero un set di soluzioni "*non dominate*" in cui nessun allineamento può essere migliorato in un parametro senza subire un peggioramento nell'altro.

Questo approccio è particolarmente sensato in ambito genomico, poiché restituisce al ricercatore un ventaglio di allineamenti biologicamente plausibili, garantendo un'esplorazione robusta dello spazio discreto generato dai gap.

4.3.4 Analisi dell'Implementazione

Il ciclo di vita del nostro algoritmo si articola in una serie di fasi rigorose.

Generazione e Valutazione Iniziale: L'algoritmo inizializza la popolazione inserendo sequenze clonate e sequenze modificate con l'inserimento stocastico di gap. Successivamente si valuta ogni sequenza restituendo una tupla di punteggi (SP, CS).

Fast Non-Dominated Sorting : Questa fase è il cuore dell'NSGA-II. La popolazione viene suddivisa in "fronti" di dominanza. Per ogni individuo p , si calcola quante soluzioni lo dominano ('domination_count') e un set di soluzioni che esso domina ('dominated_set').

- Un individuo domina un altro se i suoi punteggi sono $\geq$ in entrambi gli obiettivi (SP, CS) e strettamente $>$ in almeno uno.
- Gli individui con $domination_count == 0$ formano il **Primo Fronte** (i migliori in assoluto).
- Rimuovendo *temporaneamente* questo fronte, si calcola il Secondo Fronte, e così via, stratificando l'intera popolazione.

Calcolo della Crowding Distance: Per prevenire il collasso della diversità genetica e mantenere soluzioni variegata, l'algoritmo calcola una metrica di "densità" per ogni individuo all'interno del proprio fronte. La *Crowding Distance* rappresenta il perimetro del cuboide formato dai due vicini più prossimi dell'individuo nello spazio degli obiettivi. Agli individui situati agli estremi del fronte (i valori massimi e minimi assoluti) viene assegnata una distanza infinita per garantirne la sopravvivenza. Per gli individui intermedi i , la distanza CD_i è calcolata esplicitamente come:

$$CD_i = \sum_{m=1}^M \frac{f_m^{(i+1)} - f_m^{(i-1)}}{f_m^{max} - f_m^{min}}$$

dove M è il numero di obiettivi (nel nostro caso 2: SP e CS), $f_m^{(i+1)}$ e $f_m^{(i-1)}$ sono i valori di fitness dei due individui adiacenti per l'obiettivo m , mentre f_m^{max} e f_m^{min} sono i valori massimi e minimi di quell'obiettivo nel fronte. Un valore di CD_i più alto indica che la soluzione si trova in una "nicchia" poco esplorata, rendendola biologicamente preziosa per la diversità genetica 10.

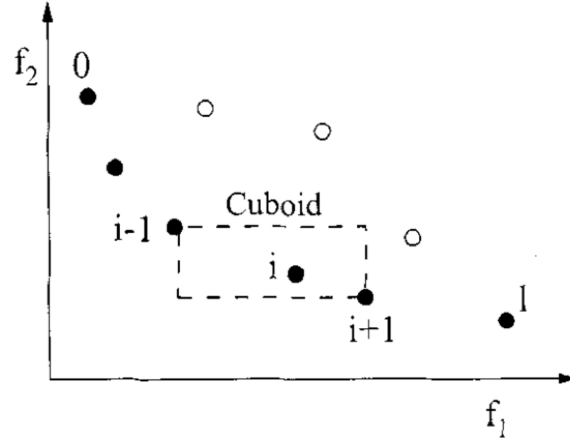


Figure 10: Visualizzazione del calcolo di Crowding-distance. I punti pieni segnati sono soluzione dello stesso fronte Non-Dominat [2]

Operatori di Selezione e Riproduzione: Una volta che la popolazione è stata valutata, a ogni individuo vengono assegnati due parametri fondamentali: il **Rango** (corrispondente al numero del suo Fronte di appartenenza, dove 1 è il migliore) e la **Crowding Distance**. A questo punto, l'algoritmo procede con l'evoluzione:

1. **Tournament Selection e Mating Pool:** La selezione degli individui destinati a riprodursi avviene tramite tornei binari ($k = 2$). Vengono estratti casualmente due candidati e il vincitore viene stabilito tramite il *Crowded Comparison Operator*, una regola gerarchica rigorosa: vince sempre l'individuo con il Rango migliore. Solo a parità di Rango (stesso fronte), vince l'individuo con la Crowding Distance maggiore. I vincitori di questi tornei vengono inseriti in una lista temporanea chiamata *mating pool*.
2. **Crossover e Mutazione Intelligente (Generazione dei Figli):** Gli individui selezionati nella *mating pool* agiscono da "genitori" e subiscono due trasformazioni fondamentali per generare la nuova prole (Q_t). La prima è l'*Horizontal Crossover*, un operatore che combina il materiale genetico scambiando intere porzioni di allineamento (righe) tra due soluzioni genitrici. La seconda trasformazione rappresenta il fulcro ibrido e innovativo dell'architettura: la **mutazione mirata tramite agente**. A differenza dei GA tradizionali, in cui la mutazione è un'alterazione puramente stocastica (es. inserimento o rimozione casuale di gap), in questo framework la mutazione agisce come un raffinato operatore di *local search*.

Il processo si articola come segue:

- L'algoritmo effettua una scansione dell'intero allineamento candidato per identificare la regione (sottomatrice) con il punteggio di fitness peggiore.
- Questa "finestra" sub-ottimale viene estratta, eventualmente paddata per rispettare le dimensioni di input attese dal modello, e convertita nello stato di un Environment per l'agente di Reinforcement Learning.
- L'agente osserva lo stato e attua una policy decisionale attiva, eseguendo step di allineamento per massimizzare il reward locale.
- La sottomatrice così "risolta" e ottimizzata dalla rete viene infine ricucita all'interno del cromosoma originale.

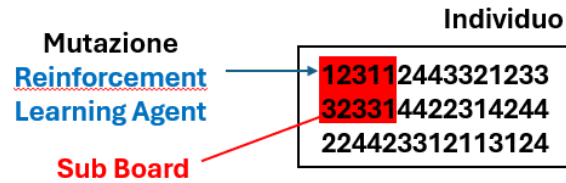


Figure 11: Visualizzazione operazione di mutazione

L'esito congiunto di queste due operazioni genera una nuova popolazione di figli numericamente pari a quella dei genitori (P_t).

3. **Selezione Elitista N+N:** Per decidere chi sopravviverà nella generazione successiva, l'algoritmo unisce i genitori (P_t) e i figli appena creati (Q_t) in un'unica super-popolazione temporanea di dimensione $2N$. Questa popolazione unita viene classificata nuovamente in Fronti tramite il *Non-Dominated Sorting* (basato unicamente sui punteggi SP e CS). La nuova generazione definitiva (di dimensione N) viene formata prelevando progressivamente individui dai fronti migliori, partendo dal Fronte 1. Se durante questo riempimento l'inclusione di un intero fronte fa superare il limite di capienza N , si utilizza la Crowding Distance come criterio di spareggio: le soluzioni di quel frammento vengono ordinate per distanza decrescente e vengono salvate solo quelle più "isolate", garantendo la massima esplorazione dello spazio delle soluzioni e scartando quelle troppo simili tra loro.

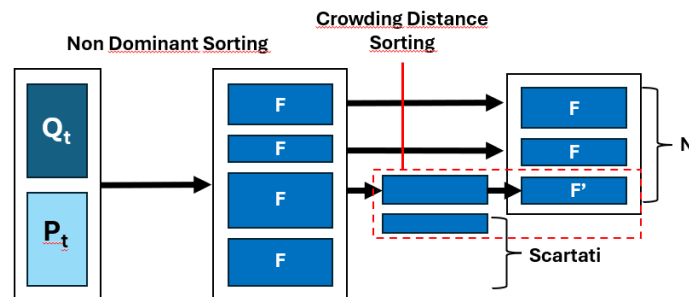


Figure 12: unione della popolazione P_t , Q_t in un'unica popolazione ordinata sulla base del Rank (Fronti F). Sulla base del sorting si prelevano gli individui appartenenti al fronte fino ad una dimensione massima N . Intra-fronte per selezione si ordina sulla base della crowding distance sorting

Selezione dell'Allineamento Definitivo (Scalarizzazione a Posteriori): La natura dell'algoritmo NSGA-II è intrinsecamente progettata per restituire un intero set di soluzioni ottimali (il Fronte di Pareto), offrendo all'utente diverse alternative di compromesso. Tuttavia, l'architettura del nostro orchestratore e i successivi benchmark richiedono in output un singolo file FASTA definitivo.

Per colmare questa discrepanza, l'estrazione della soluzione migliore dal Fronte 1 finale avviene tramite una **funzione di scalarizzazione** basata su una somma pesata:

$$\text{combined_eval} = SP + (CS \times 100)$$

L'adozione di questa specifica formulazione si è resa necessaria per due ragioni fondamentali:

- **Normalizzazione delle metriche:** La metrica SP (Sum-of-Pairs) genera tipicamente valori su scale di grandezza molto elevate (ordine delle migliaia), mentre il CS (Column Score) produce valori dimensionalmente inferiori (spesso frazionari o nell'ordine delle decine).
- **Prevenzione del collasso degli obiettivi:** Senza l'introduzione del moltiplicatore 100, l'obiettivo SP cannibalizzerebbe matematicamente la scelta finale, vanificando di fatto l'ottimizzazione multiobiettivo. Questo bilanciamento garantisce invece la selezione programmatica del compromesso biologico più coerente.

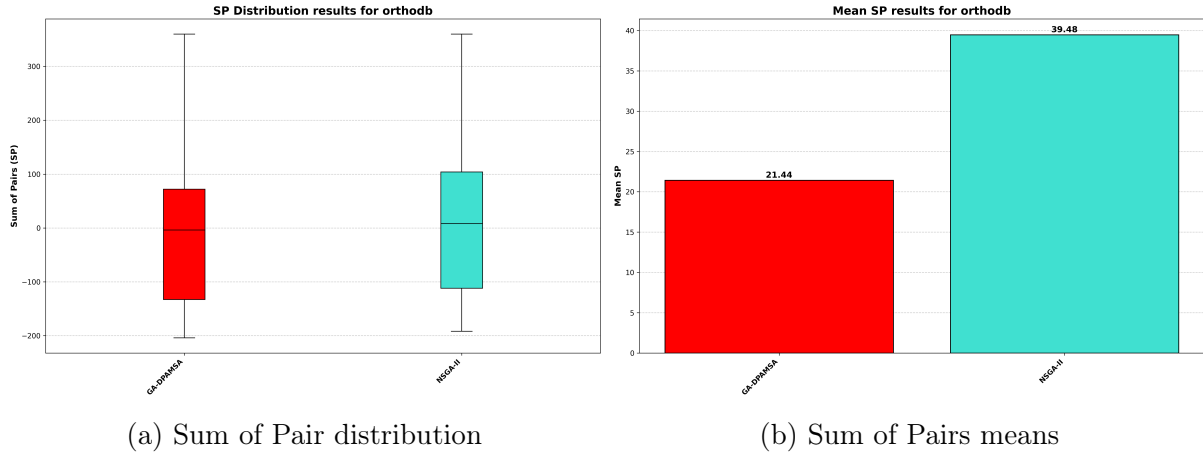


Figure 13: Comparazione dei modelli GA-DPAMSA (sinsitra) e NSGA-II(destra) per metriche di SP.

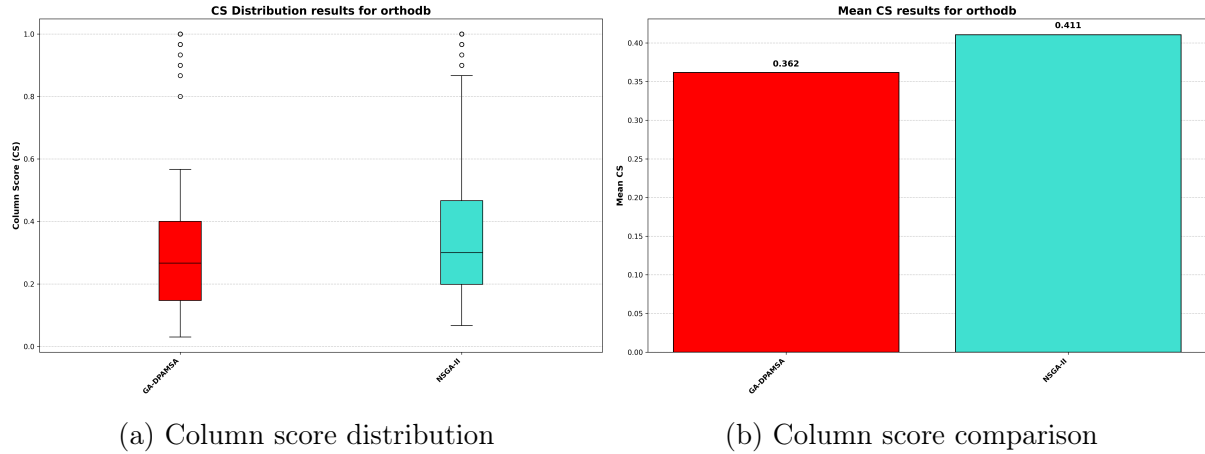


Figure 14: Comparazione dei modelli GA-DPAMSA (sinistra) e NSGA-II(destra) per metriche di CS.

4.3.5 Confronto Prestazionale

Il passaggio dall'algoritmo genetico di baseline alla nuova architettura basata su NSGA-II segna un netto cambio di paradigma nella risoluzione del Multiple Sequence Alignment. Come si evince dall'analisi delle distribuzioni e delle medie dei punteggi (Figure 13 e 14), l'approccio originale pagava il prezzo di una selezione basata sull'unione forzata degli indici: le soluzioni tendevano a polarizzarsi verso gli estremi, generando un'elevata varianza e favorendo allineamenti che massimizzavano una metrica a discapito della totale compromissione dell'altra.

Al contrario, l'introduzione del **Fast Non-Dominated Sorting e della Crowding Distance** ha prodotto un duplice beneficio empirico:

1. In primo luogo, ha stabilizzato l'esplorazione dello spazio di ricerca, prevenendo la convergenza prematura e riducendo drasticamente le fluttuazioni anomale nei punteggi medi tra le generazioni.
2. In secondo luogo, trattando SP e CS come obiettivi ortogonali e paritari fino al momento dell'estrazione finale, l'orchestratore riesce a mantenere viva un'intera frontiera di soluzioni ibride.

I grafici dimostrano come l'NSGA-II non solo elevi la qualità media globale della popolazione rispetto alla baseline, ma garantisca anche una distribuzione dei punteggi molto più densa e coerente. L'applicazione finale della funzione di scalarizzazione a posteriori agisce infine come un filtro di precisione, assicurando che l'allineamento restituito in output non sia un semplice picco statistico isolato, ma il miglior compromesso biologico matematicamente validato all'interno del Fronte di Pareto. In sintesi, il nuovo orchestratore risolve le criticità strutturali della versione precedente, trasformando una ricerca stocastica e sbilanciata in un processo di ottimizzazione mirato, stabile e biologicamente consistente.

5 Risultati

5.1 Analisi Comparativa delle Architetture Sviluppate

A conclusione del processo di sviluppo e ottimizzazione, è stata condotta un'analisi comparativa per valutare le reali prestazioni di tutte le architetture implementate in questo studio. L'obiettivo di questa fase è quantificare l'impatto delle singole innovazioni (algoritmi genetici, orchestrazione multi-obiettivo, agenti di Reinforcement Learning) rispetto alla configurazione di partenza.

La valutazione è stata effettuata sempre su dati ortholghi (assicurandoci di considerare dati che i modelli non abbiano visto durante la fase di addestramento) e normalizzato, misurando sia il *Sum of Pairs* (SP) che il *Column Score* (CS). I risultati medi e le distribuzioni prestazionali, riassunti nelle figure in 15 e 16, delineano una chiara traiettoria evolutiva dei nostri modelli.

5.1.1 Dalla Baseline all'Ottimizzazione Multi-Obiettivo

Osservando le metriche, si nota immediatamente come l'approccio originale **DPAMSA** presenti i limiti più marcati, registrando i valori medi peggiori in assoluto: un CS quasi nullo pari a **0.076** e un SP profondamente negativo di **-302.44**. Questi risultati confermano la difficoltà dell'agente base di gestire allineamenti multipli complessi senza una corretta orchestrazione globale.

L'introduzione della componente genetica ha portato i primi sostanziali miglioramenti. L'architettura **GA-DPAMSA** ha risollevato il CS a **0.215** e migliorato l'SP a **-82.48**. Un risultato leggermente migliore si ottiene andando a sostituire con il nuovo orchestratore presentato **NSGA-II** (**CS: 0.226, SP: -77.48**). Sebbene queste architetture dimostrino l'efficacia della ricerca globale nello sfuggire agli ottimi locali, la mancanza di una supervisione mirata a livello locale impedisce di raggiungere punteggi positivi nel Sum of Pairs.

5.1.2 Confronto Reinforcement Learning

Il cambio di paradigma si osserva introducendo agenti di Reinforcement Learning avanzati. L'utilizzo di algoritmi di policy gradient come **PPO** e **GRPO** ha prodotto un netto balzo in avanti. È interessante notare come entrambi i modelli abbiano restituito prestazioni empiriche identiche su questo benchmark, stabilizzandosi su un CS di **0.354** e ottenendo per la prima volta un SP positivo di **19.04**.

L'unione dei due mondi ha generato l'architettura ibrida **NSGA-II-DCNNM**. In questa configurazione la DCNN opera come *local search* sulla finestra sub-ottimale, il CS raggiunge lo **0.411** e l'SP sale a **39.48** confermando la sinergia vincente tra ottimizzazione paretiana e Reinforcement Learning.

5.1.3 L'Egemone nel Benchmark: DCNNMSA

Il vincitore assoluto di questa fase di testing interno è risultata l'architettura **DCNN-MSA**. Distaccando tutte le altre varianti analizzate, questo modello ha registrato un **Column Score medio di 0.441** e ha massimizzato il **Sum of Pairs portandolo al picco di 46.24**.

Come si evince anche dai boxplot delle distribuzioni, DCNNMSA non solo eleva il tetto massimo delle performance (SP e CS), ma garantisce anche una notevole robustezza, limitando la varianza degli allineamenti fallimentari. Questo risultato sancisce che l'architettura DCNNMSA rappresenta il miglior compromesso tra capacità di apprendimento dell'agente e corretta gestione dell'ambiente discreto dell'allineamento.

Alla luce di questi risultati, DCNNMSA e la variante ibrida NSGA-II-DCNN si candidano come i modelli di punta del nostro progetto, pronti per essere messi alla prova nel confronto finale contro i software di allineamento multiplo attualmente considerati lo stato dell'arte nella comunità bioinformatica.

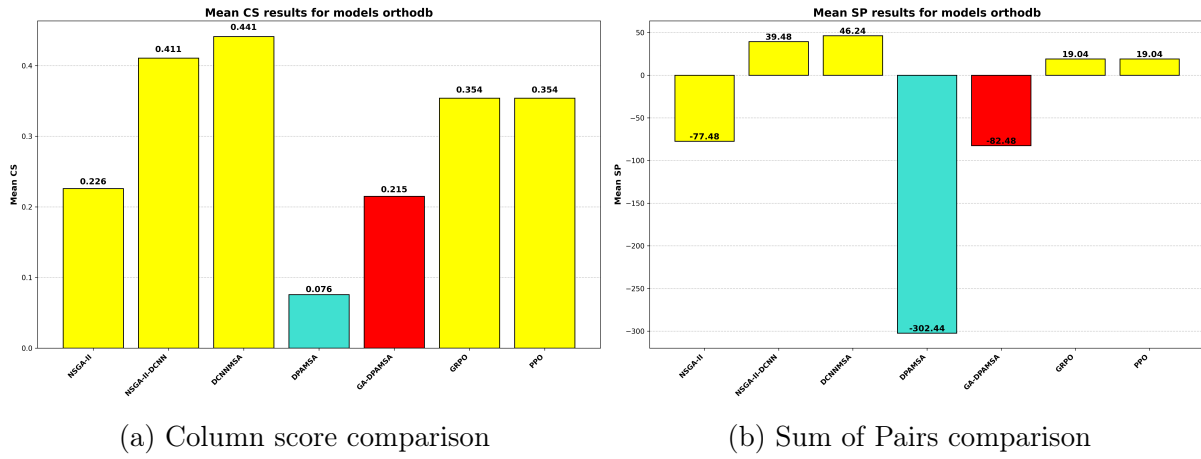
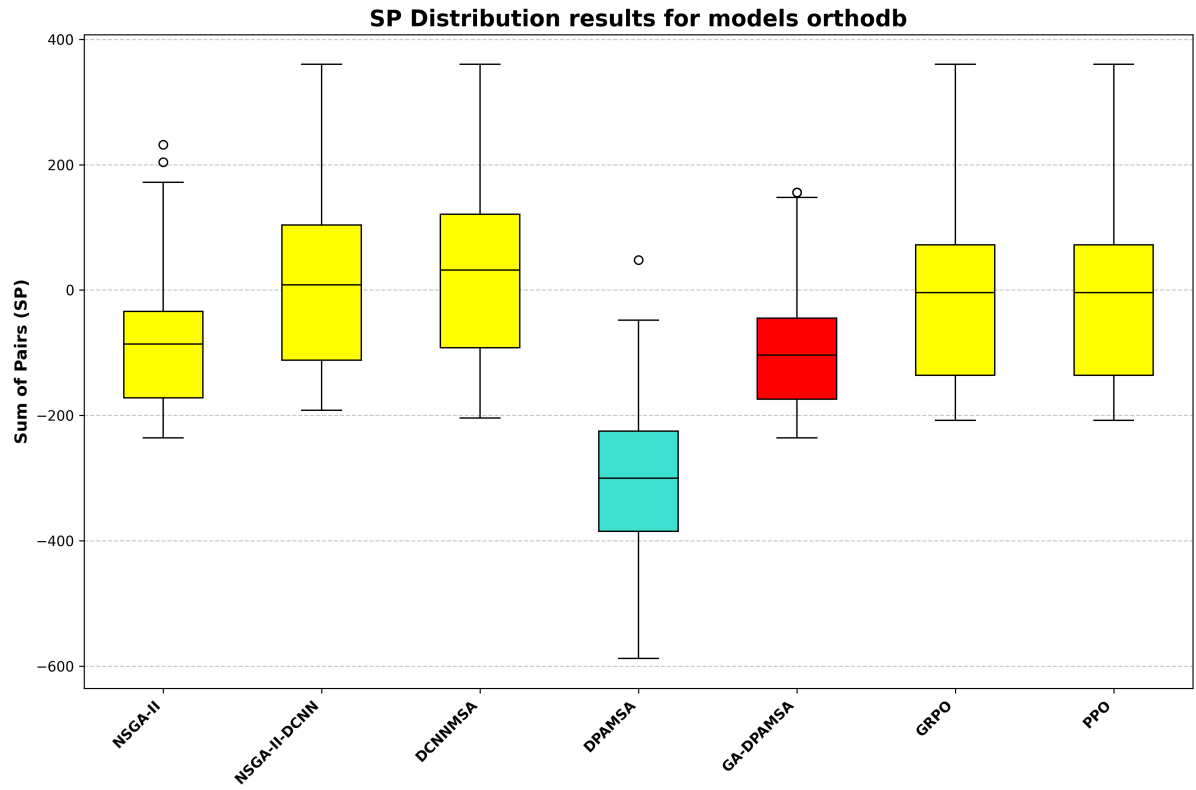
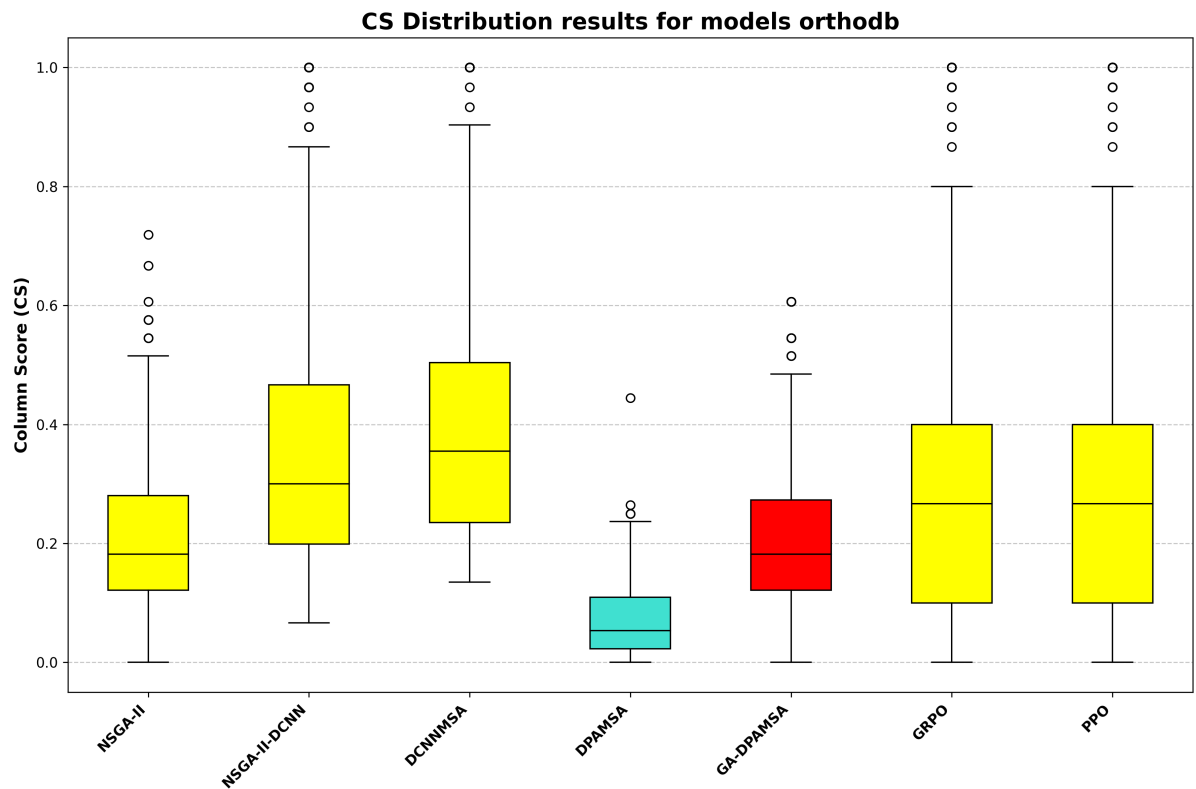


Figure 15: Comparazione dei valori medi per Column Score (a) e Sum of Pairs (b)



(a) Distribuzione Sum of Pairs



(b) Distribuzione Column Score

Figure 16: Comparazione CS (a) e SP (b) tra tutti i modelli sviluppati (in giallo) e quelli di base line (in rosso e turchese)

5.2 Confronto con lo Stato dell'Arte

Per validare definitivamente la bontà degli approcci proposti, i modelli di punta emersi dalla precedente analisi (DCNNMSA e la variante NSGA-II-DCNN) e tutti gli altri sono stati messi a confronto con alcuni dei software di allineamento multiplo più consolidati e utilizzati dalla comunità scientifica: ClustalOmega, ClustalW, MAFFT, MSAPROBS, MUSCLE e PASTA. Il benchmark è stato condotto sul medesimo dataset di sequenze ortologhe, valutando le metriche di Column Score (CS) e Sum of Pairs (SP) come possiamo vedere dalle figure in 17.

5.2.1 Analisi Column Score (CS)

Osservando i risultati relativi al Column Score, emerge chiaramente come l'architettura **DCNNMSA** si inserisca con forza nella fascia medio-alta dei tool professionali. Con un CS medio di **0.441**, il nostro modello **supera software ampiamente diffusi come MAFFT (0.437), MUSCLE (0.421) e PASTA (0.366)**.

La variante ibrida **NSGA-II-DCNN (0.411)** si difende altrettanto bene, posizionandosi a ridosso di MUSCLE e mantenendo un netto vantaggio su PASTA. Sebbene il primato per questa metrica resti appannaggio di algoritmi storici e iperottimizzati come MSAPROBS (0.491), ClustalW (0.461) e ClustalOmega (0.451), il distacco di DCNNMSA dalla vetta risulta estremamente contenuto.

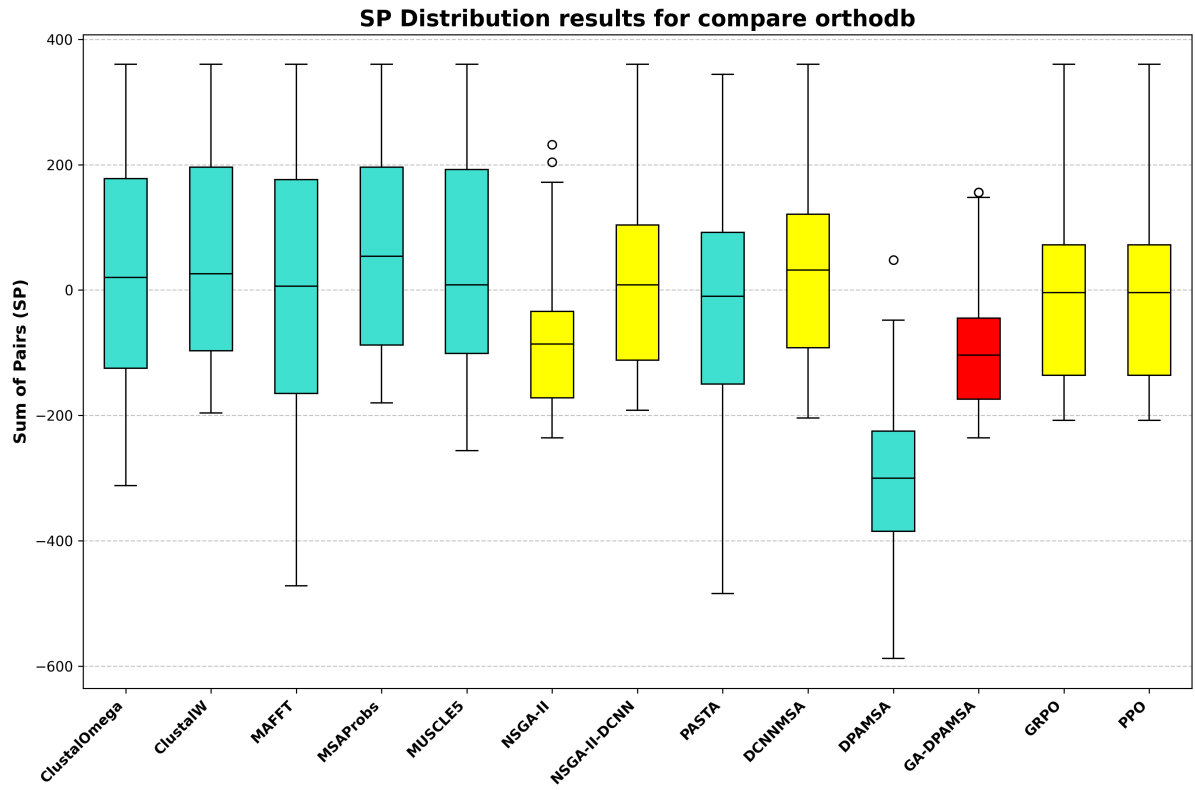
5.2.2 Analisi del Sum of Pairs (SP):

Il confronto sul Sum of Pairs conferma e rafforza le potenzialità della nostra architettura. In questo contesto, **DCNNMSA** ottiene un punteggio di **46.24**, riuscendo a **sorpassare non solo PASTA (-8.58) e MAFFT (12.40), ma persino ClustalOmega (42.68)**.

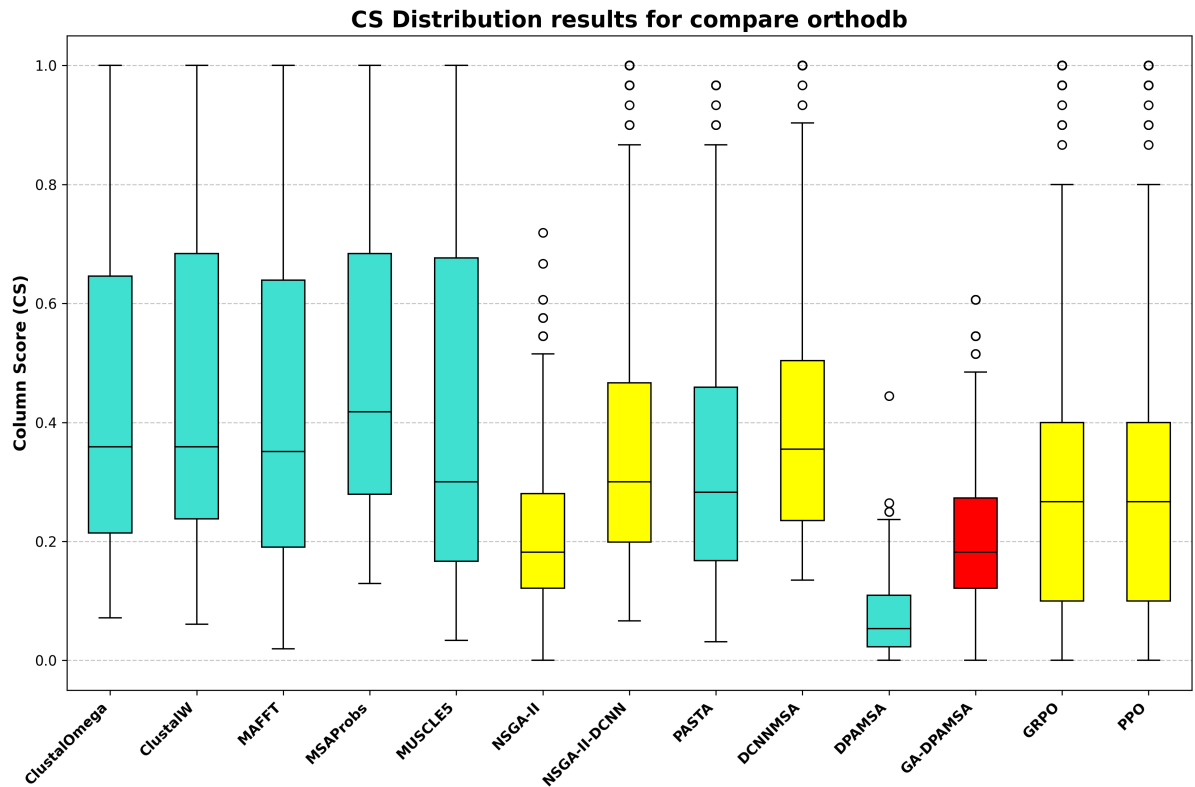
Anche l'architettura **NSGA-II-DCNN (39.48)** dimostra una notevole solidità, distaccando ampiamente MAFFT e PASTA. Le posizioni di vertice per l'SP sono occupate da MSAPROBS (73.08), ClustalW (59.56) e MUSCLE (50.28).

5.3 Considerazioni Finali

In sintesi, i dati empirici dimostrano che l'approccio basato su Deep Reinforcement Learning (incarnato in DCNNMSA) non è solo un puro esercizio teorico, ma un'alternativa concreta e competitiva. Riuscire a superare le performance di tool che rappresentano lo standard de facto (come MAFFT e PASTA, o ClustalOmega nel caso dell'SP) prova la robustezza della formulazione del problema e l'efficacia del training. Mentre gli algoritmi tradizionali (MsaProbs e ClustalW) mantengono l'assoluto primato, le architetture presentate in questo studio pongono basi solidissime per gli sviluppi futuri, dimostrando che l'intelligenza artificiale può competere efficacemente nel dominio dell'allineamento multiplo di sequenze.



(a) Distribuzione Column Score



(b) Distribuzione Sum of Pairs

Figure 17: Comparazione CS (a) e SP (b) tra tutti i modelli sviluppati (in giallo) e quelli di base line (in rosso e turchese) e ulteriori modelli allo stato dell'arte (in turchese)

6 Conclusioni e Sviluppi Futuri

6.1 Conclusioni

Il presente lavoro ha affrontato la complessa sfida dell’Allineamento Multiplo di Sequenze (MSA) esplorando il potenziale dell’Intelligenza Artificiale, e in particolare del Deep Reinforcement Learning (DRL), come alternativa e potenziamento rispetto ai tradizionali algoritmi euristici. Partendo dai limiti strutturali evidenziati nell’architettura di baseline GA-DPAMSA, abbiamo condotto una revisione sistematica dell’intero framework, agendo su tre direttrici fondamentali: l’ingegneria dei dati, il design delle reti neurali e l’orchestrazione evolutiva.

Il primo traguardo cruciale è stato il passaggio da un dominio puramente sintetico a uno biologicamente realistico. La costruzione di una pipeline di data engineering scalabile e automatizzata, basata sull’estrazione di sequenze ortologhe dal database OrthoDB, ha permesso di addestrare i nostri agenti su pattern di conservazione e divergenza evolutiva reali, risolvendo il problema della memorizzazione del “rumore” stocastico e garantendo un’effettiva capacità di generalizzazione (zero-shot).

Sul fronte architetturale, l’abbandono di meccanismi di attention computazionalmente onerosi in favore delle *Dilated Convolutional Neural Networks* (DCNN), unito alla gestione dello spazio esponenziale delle azioni tramite *Action Branching* (Dueling DQN), ha permesso di creare agenti reattivi, stabili e veloci. Parallelamente, l’implementazione dell’NSGA-II ha rivoluzionato la ricerca globale, sostituendo un bilanciamento forzato con una vera esplorazione del Fronte di Pareto, in cui il Reinforcement Learning funge da raffinato operatore di ricerca locale.

Il culmine di questo sforzo ingegneristico si è concretizzato nell’architettura **DCNN-MSA**. Come ampiamente dimostrato nei benchmark finali, questo modello ha non solo annientato le performance della baseline, ma è riuscito a superare in accuratezza (Sum of Pairs e Column Score) software storici e iper-ottimizzati come MAFFT, MUSCLE, PASTA e ClustalOmega. Questo risultato possiede una forte valenza scientifica: dimostra inequivocabilmente che le architetture basate su Deep Reinforcement Learning sono oggi mature per competere ai massimi livelli nello stato dell’arte della bioinformatica computazionale.

6.2 Sviluppi Futuri

Sebbene i risultati ottenuti da DCNNMSA e NSGA-II-DCNN segnino un punto di svolta, il framework sviluppato pone solide basi per molteplici traiettorie di ricerca e ottimizzazione futura:

- **Scalabilità Dimensionale della Finestra Operativa:** Attualmente, gli agenti operano su sottomatrici di dimensioni ridotte (es. 3×30). Il prossimo passo consisterà nell'addestrare modelli capaci di gestire finestre di contesto nettamente superiori (es. 10×100). Ciò richiederà un bilanciamento tra la VRAM disponibile in fase di training e l'adozione di architetture neurali ancora più profonde.
- **Reward Biologicamente Informato:** La funzione di ricompensa attuale è strettamente legata alle metriche di valutazione SP e CS. Un'evoluzione naturale prevederà l'integrazione di matrici di sostituzione amminoacidica o nucleotidica (es. BLOSUM o PAM) direttamente nel calcolo del reward dell'agente, forzandolo a preferire allineamenti filogeneticamente più coerenti piuttosto che matematicamente esatti.
- **Integrazione di Attention Lineari:** Avendo risolto il collo di bottiglia quadratico della baseline passando alle convoluzioni, si potrebbe valutare la re-introduzione di meccanismi di *Self-Attention* moderni (come Linformer o Performer), che scalano linearmente. Un'architettura ibrida CNN-Transformer potrebbe catturare sia i pattern locali (tramite convoluzioni) sia le dipendenze a lunghissimo raggio all'interno dell'intero filamento di DNA.
- **Orchestrazione Multi-Agente Gerarchica:** Si potrebbe superare del tutto la dipendenza dall'algoritmo genetico demandando l'intero processo a un'architettura gerarchica (Hierarchical RL). Un "Agente Manager" (globale) avrebbe il compito di scansionare l'allineamento per individuare le macro-regioni sub-ottimali, delegandone la risoluzione a grana fine all'agente DCNN (locale), realizzando così un framework end-to-end guidato puramente da reti neurali.
- **Allineamento Strutturale (Structure-Aware MSA):** Sfruttando le recenti rivoluzioni nel campo della predizione proteica (es. AlphaFold), i futuri modelli potrebbero accettare in input non solo la sequenza monodimensionale, ma anche embeddings rappresentanti la struttura 3D predetta delle molecole. In questo modo, l'agente imparerebbe a inserire gap in modo da preservare non solo l'identità nucleotidica, ma anche la stabilità dei domini strutturali.

References

- [1] Margaret Dayhoff, R Schwartz, and B Orcutt. 22 a model of evolutionary change in proteins. *Atlas of protein sequence and structure*, 5:345–352, 1978.
- [2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [3] Edo Dotan, Yonatan Belinkov, Oren Avram, Elya Wygoda, Noa Ecker, Michael Alburquerque, Omri Keren, Gil Loewenthal, and Tal Pupko. Multiple sequence alignment as a sequence-to-sequence learning problem. In *The Eleventh International Conference on Learning Representations*, 2023.
- [4] Edo Dotan, Elya Wygoda, Noa Ecker, Michael Alburquerque, Oren Avram, Yonatan Belinkov, and Tal Pupko. Betaalign: a deep learning approach for multiple sequence alignment. *Bioinformatics*, 41(1):btaf009, 2025.
- [5] Gianni D’Angelo and Francesco Palmieri. Gga: A modified genetic algorithm with gradient-based local search for solving constrained optimization problems. *Information Sciences*, 547:136–162, 2021.
- [6] Robert C Edgar. Muscle: multiple sequence alignment with high accuracy and high throughput. *Nucleic acids research*, 32(5):1792–1797, 2004.
- [7] Osamu Gotoh. An improved algorithm for matching biological sequences. *Journal of molecular biology*, 162(3):705–708, 1982.
- [8] Steven Henikoff and Jorja G Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the national academy of sciences*, 89(22):10915–10919, 1992.
- [9] Reza Jafari, Mohammad Masoud Javidi, and Marjan Kuchaki Rafsanjani. Using deep reinforcement learning approach for solving the multiple sequence alignment problem. *SN Applied Sciences*, 1(6):592, 2019.
- [10] Kazutaka Katoh, Kazuharu Misawa, Kei-ichi Kuma, and Takashi Miyata. Mafft: a novel method for rapid multiple sequence alignment based on fast fourier transform. *Nucleic acids research*, 30(14):3059–3066, 2002.
- [11] Wentian Li. The study of correlation structures of dna sequences: a critical review. *Computers & chemistry*, 21(4):257–271, 1997.
- [12] Yuhang Liu, Hao Yuan, Qiang Zhang, Zixuan Wang, Shuwen Xiong, Naifeng Wen, and Yongqing Zhang. Multiple sequence alignment based on deep reinforcement learning with self-attention and positional encoding. *Bioinformatics*, 39(11):btad636, 2023.
- [13] Ioan-Gabriel Mircea, Gabriela Czibula, and Maria-Iuliana Bocicor. A q-learning approach for aligning protein sequences. In *2015 IEEE International Conference on Intelligent Computer Communication and Processing (ICCP)*, pages 51–58. IEEE, 2015.

- [14] Saul B Needleman and Christian D Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of molecular biology*, 48(3):443–453, 1970.
- [15] Cédric Notredame, Desmond G Higgins, and Jaap Heringa. T-coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of molecular biology*, 302(1):205–217, 2000.
- [16] Samantha Petti, Nicholas Bhattacharya, Roshan Rao, Justas Dauparas, Neil Thomas, Juannan Zhou, Alexander M Rush, Peter Koo, and Sergey Ovchinnikov. End-to-end learning of multiple sequence alignments with differentiable smith–waterman. *Bioinformatics*, 39(1):btac724, 2023.
- [17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [19] Fabian Sievers, Andreas Wilm, David Dineen, Toby J Gibson, Kevin Karplus, Weizhong Li, Rodrigo Lopez, Hamish McWilliam, Michael Remmert, Johannes Söding, et al. Fast, scalable generation of high-quality protein multiple sequence alignments using clustal omega. *Molecular systems biology*, 7:539, 2011.
- [20] Arash Tavakoli, Fabio Pardo, and Petar Kormushev. Action branching architectures for deep reinforcement learning. In *Proceedings of the aaai conference on artificial intelligence*, volume 32, 2018.
- [21] Julie D Thompson, Desmond G Higgins, and Toby J Gibson. Clustal w: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic acids research*, 22(22):4673–4680, 1994.
- [22] Julie D. Thompson, Frédéric Plewniak, and Olivier Poch. Balibase: a benchmark alignment database for the evaluation of multiple alignment programs. *Bioinformatics (Oxford, England)*, 15(1):87–88, 1999.
- [23] Lusheng Wang and Tao Jiang. On the complexity of multiple sequence alignment. *Journal of computational biology*, 1(4):337–348, 1994.
- [24] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, New York, New York, USA, 20–22 Jun 2016. PMLR.
- [25] Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. *arXiv preprint arXiv:1511.07122*, 2015.

- [26] Rocco Zaccagnino, Andrea Aceto, Gerardo Benevento, Gerardo Frino, Nicola Frugieri, Delfina Malandrino, Alessia Ture, and Gianluca Zaccagnino. Scalable multiple sequence alignment via genetic algorithms and localized deep reinforcement learning agents. In *2025 IEEE 25th International Conference on Bioinformatics and Bioengineering (BIBE)*, pages 363–367. IEEE, 2025.
- [27] Yongqing Zhang, Qiang Zhang, Yuhang Liu, Meng Lin, and Chunli Ding. Multiple sequence alignment based on deep q network with negative feedback policy. *Computational Biology and Chemistry*, 101:107780, 2022.